

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет «Школа информатики, физики и технологий»

[Фамилия Имя Отчество автора]

**Адаптивные топологии мульти-агентных
систем больших языковых моделей с включением
человека в контур управления**

Выпускная квалификационная работа

бакалаврская работа

по направлению подготовки 01.03.02

«Прикладная математика и информатика»

образовательная программа

«Прикладной анализ данных и искусственный интеллект»

Рецензент

Руководитель

[уч. степень, звание]

[уч. степень, звание]

[И.О. Фамилия]

[И.О. Фамилия]

Санкт-Петербург, 2026

Оглавление

Аннотация	4
Abstract	4
Ключевые слова	5
Введение	6
1 Аналитический обзор литературы	11
2 Постановка задачи и методология исследования	20
3 Архитектура программной системы	31
4 Экспериментальное исследование	41
5 Анализ результатов и обсуждение	52
Авторский вклад	58
Заключение	61
Библиографический список	64
А Функциональная схема системы	67
Б Глоссарий	70
В Состав программного репозитория	74
Г Открытый исходный код	76

Аннотация

В работе проведена количественная оценка влияния выбора коммуникационной топологии мульти-агентной системы, механизма ее адаптации в процессе решения задачи и позиции человека в контуре управления на эффективность решения задач разных классов. Объектом исследования выступают мульти-агентные системы на основе больших языковых моделей; предметом — коммуникационная топология таких систем, механизм ее динамической адаптации и роль человека в графе обмена сообщениями. Разработан и опубликован под лицензией MIT программный фреймворк, объединяющий пять статических топологий, адаптивную мета-топологию, пять ролей человека и три режима маршрутизации. Воронка из четырех экспериментов с двумя кросс-семейными подтверждающими прогонами включает 5175 прогонов на четырех корпусах задач. Получены положительные ответы на два из четырех исследовательских вопросов; на два оставшихся — отрицательные. Рядом с отрицательными ответами обнаружены две сопутствующие находки, которые при кросс-семейной валидации также не подтверждаются. Центральный вывод работы — кросс-семейная неустойчивость адаптивных политик маршрутизации, требующая обязательной валидации не менее чем на двух семействах весов рабочего агента.

Abstract

The thesis provides a quantitative assessment of how the choice of communication topology, its runtime adaptation, and the position of the human in the control loop affect the effectiveness of multi-agent large-language-model systems across task classes. The object of study is multi-agent systems built on large language models; the subject — their communication topology, the mechanism of its dynamic adaptation, and the role of the human in the agent message graph. A multi-agent framework was developed and published under the MIT license, in-

tegrating five static topologies, an adaptive meta-topology, five human roles, and three routing modes. A funnel of four experiments with two cross-family confirmation runs covers 5175 runs across four task corpora. Two of the four research questions receive positive answers; the remaining two are rejected. Alongside the negative answers, two related side findings are identified and likewise fail under cross-family validation. The central finding is the cross-family non-invariance of adaptive routing policies, which requires mandatory validation on at least two worker model families.

Ключевые слова

мульти-агентные системы, большие языковые модели, коммуникационная топология, адаптивная маршрутизация, человек в контуре управления, оракул адаптации, кросс-семейная валидация, Парето-оптимизация, бутстрап-оценивание, оркестрация LLM-агентов.

Keywords: multi-agent systems, large language models, communication topology, adaptive routing, human-in-the-loop, adaptation oracle, cross-family validation, Pareto optimization, bootstrap inference, LLM agent orchestration.

Введение

Актуальность работы. В период с 2023 по 2026 годы применение больших языковых моделей (от англ. Large Language Models, LLM) перешло от одиночных диалоговых сценариев к кооперативной работе нескольких автономных агентов, образующих мульти-агентную систему (от англ. Multi-Agent System, MAS). Известные открытые фреймворки — AutoGen, ChatDev, MetaGPT, Magentic-One — демонстрируют, что разделение труда между специализированными агентами повышает качество решения сложных задач, недоступных одиночной модели. Однако каждый такой фреймворк жестко фиксирует структуру коммуникации между агентами на этапе проектирования. Систематических данных о том, какая структура является оптимальной для каждого класса задач, в открытой литературе нет. Параллельно растет интерес к интеграции человека в контур управления мульти-агентными системами, но и здесь существующие работы ограничиваются единичными паттернами включения человека без сравнения их эффективности. Сочетание двух этих обстоятельств — интенсивного роста мульти-агентных архитектур и отсутствия систематических знаний о выборе их топологии и позиции человека — определяет научную и прикладную актуальность настоящей работы.

Научная новизна. В настоящей работе впервые проведено прямое сравнение пяти базовых коммуникационных топологий (звезда, цепочка, полносвязная, дебатная, иерархическая) на едином программном стенде, на одной языковой модели и на одной выборке задач из четырех различных классов. Предложена и реализована мета-топология, переключающая активную базовую конфигурацию в зависимости от текущей фазы решения и наблюдаемых сигналов агентов. Введена таксономия из пяти специфичных для адаптивных мульти-агентных систем метрик — N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle} , — позволяющая количественно характеризовать поведение мета-топологии. Впервые сформулирована и экспериментально проверена матрица совместимости пяти ролей человека в контуре управления с пятью базовыми топологиями.

Предмет исследования. Коммуникационная топология мульти-агентной

системы больших языковых моделей, механизм ее динамической адаптации в процессе решения задачи, а также позиция человека в графе обмена сообщениями между агентами.

Объект исследования. Мульти-агентные системы на основе больших языковых моделей, решающие сложные задачи различных классов в кооперации со специализированными агентами и участником-человеком.

Цель работы. Количественная оценка влияния выбора коммуникационной топологии мульти-агентной системы, механизма ее адаптации и позиции человека в контуре управления на эффективность решения задач разных классов с одновременным учетом качества решения, его стоимости и времени выполнения.

Аналитический обзор предлагаемых решений. В публикациях 2022–2026 годов выделяется несколько устойчивых направлений развития мульти-агентных систем больших языковых моделей. Архитектурные фреймворки (AutoGen, ChatDev, MetaGPT) фиксируют топологию на этапе проектирования и не сравнивают альтернативные структуры между собой. Работы об автоматическом проектировании коммуникационных графов (G-Designer, GPTSwarm) подбирают оптимальную структуру под конкретную задачу однократно и не пересматривают ее в процессе решения. Работы по динамической адаптации (DyLAN, AgentVerse) изменяют состав команды агентов, но не саму структуру связей. Вопрос включения человека в контур управления мульти-агентными системами освещен фрагментарно, без сравнительной количественной оценки различных ролей. Подробный анализ всех направлений приведен в главе 1, в частности в сравнительной таблице 1.1 тринадцати методов по семи критериям.

Практическая значимость. Результаты работы предоставляют разработчикам мульти-агентных систем количественные данные о том, какая топология эффективнее для каждого из четырех рассмотренных классов задач, и при каких условиях имеет смысл включать в систему адаптивное переключение топологий. Программная инфраструктура исследования реализована в виде открытого фреймворка ATM (от англ. Adaptive Topologies for Multi-

agent systems — адаптивные топологии для мульти-агентных систем), полный исходный код которого размещен в репозитории на платформе GitHub. Фреймворк допускает повторное использование и расширение другими исследователями и практиками. Сформулированные в работе правила выбора топологии и роли человека под класс задачи могут применяться при проектировании интеллектуальных ассистентов в программировании, аналитике и принятии решений.

Теоретическая значимость. Введенная в работе таксономия адаптивно-специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle}) позволяет количественно характеризовать поведение динамически перестраиваемых мульти-агентных систем и сравнивать их между собой по единой шкале. Формализация мульти-агентной системы как кортежа из шести компонентов с зависящей от фазы коммуникационной структурой расширяет существующую теоретическую базу. Эмпирическая верификация гипотезы о том, что разные фазы решения задачи требуют разных коммуникационных топологий, дает обоснование для дальнейших теоретических работ по теории адаптивных распределенных систем интеллектуальных агентов.

Методы исследования. В работе применен комплекс взаимодополняющих методов. Аналитический обзор литературы с критериями отбора по ключевым словам в открытых базах публикаций. Формализация мульти-агентной системы средствами теории графов и теории конечных автоматов. Программная реализация распределенной системы агентов на языке Python с использованием библиотеки LangGraph для оркестрации. Контролируемый эксперимент по схеме предварительной регистрации гипотез и порогов улучшения. Статистическая обработка результатов методами дисперсионного анализа, парных сравнений с поправкой Бенджамини–Хохберга на множественное сравнение, расчета коэффициента эффекта Коэна и непараметрического бутстрап-оценивания доверительных интервалов. Оракульный референс лучшей статической топологии для каждого корпуса (per-task best-of) для построения верхней границы качества адаптации. Имитационное моделирование участника-человека средствами языковой модели с фиксированной для

каждой роли подсказкой, обеспечивающее изоляцию вклада роли человека в графе агентов.

Задачи работы. Для достижения сформулированной выше цели в работе решаются следующие задачи.

1. Провести аналитический обзор работ 2022–2026 годов по мульти-агентным системам больших языковых моделей, коммуникационным топологиям и интеграции человека в контур управления, выделить исследовательскую лакуну.
2. Сформулировать формальную модель мульти-агентной системы как кортежа из множества агентов, множества ребер коммуникации, множества ролей агентов и человека, задачи и множества фаз решения.
3. Разработать и реализовать программный фреймворк, в котором пять базовых топологий и адаптивная мета-топология реализованы в единой архитектуре на основе библиотеки LangGraph.
4. Спроектировать систему оценочных метрик, включающую базовые показатели качества, стоимости и времени, а также специфические для адаптивной мета-топологии характеристики переключений, заблокированных решений маршрутизатора, стоимостного вклада маршрутизатора, доли регрессий качества и разрыва до верхнего потолка адаптации.
5. Провести воронку из четырех экспериментов и подтверждающего прогона на четырех классах задач — программирование, математические рассуждения, творческая генерация, анализ данных — с предварительно зафиксированными гипотезами и порогами улучшения.
6. Выполнить статистический анализ результатов и сформулировать ответы на четыре исследовательских вопроса (от англ. Research Questions, RQ), описанных в разделе 2.7.
7. Выработать практические рекомендации по выбору топологии и роли человека в зависимости от класса задачи.

Структура работы. Работа состоит из введения, пяти глав основного содержания, отдельного раздела авторского вклада, заключения, библиогра-

фического списка и четырех приложений (А, Б, В, Г). Глава 1 содержит аналитический обзор литературы и определяет исследовательскую лауну. Глава 2 формализует объект исследования, вводит шесть рассматриваемых топологий, пять ролей человека, систему оценочных метрик, обосновывает выбор корпуса задач и описывает дизайн экспериментальной воронки. Глава 3 описывает архитектуру программной системы, реализующей сформулированный метод. Глава 4 представляет результаты четырех основных экспериментов и подтверждающего прогона на дополнительных семействах моделей. Глава 5 анализирует полученные результаты, дает ответы на четыре исследовательских вопроса и обсуждает угрозы валидности. Раздел авторского вклада явно фиксирует оригинальные элементы работы. Заключение подводит итоги и формулирует направления дальнейших исследований. Приложения содержат функциональную схему системы, глоссарий, краткое описание состава программного репозитория и ссылку на открытый исходный код.

В работе насчитывается пять глав основного содержания общим объемом 40 страниц, четыре приложения, 12 таблиц, 9 рисунков, 6 формул и 44 источника библиографического списка.

1 Аналитический обзор литературы

Настоящая глава представляет аналитический обзор работ, на стыке которых формируется предметная область исследования. Обзор охватывает пять направлений — архитектуру мульти-агентных систем на основе больших языковых моделей, коммуникационные топологии агентов, механизмы их адаптации, безопасность и устойчивость топологий, а также интеграцию человека в контур управления мульти-агентными системами. Глава завершается сравнительной таблицей основных методов и определением исследовательской лакуны, заполнение которой составляет содержательный вклад настоящей работы.

Методология обзора. В рассматриваемую выборку включены работы с 2022 по 2026 годы, отобранные по ключевым словам *multi-agent LLM*, *agent topology*, *LLM debate*, *human-in-the-loop multi-agent* в базах публикаций arXiv, ACL Anthology, ICLR, NeurIPS, ICML. Из первоначального списка из более чем шестидесяти работ оставлены те, которые либо предлагают новую архитектуру взаимодействия агентов, либо систематически анализируют свойства существующих архитектур. Работы, посвященные исключительно обучению одиночных моделей или классическому планированию в дискретных средах, в обзор не включены.

1.1 Мульти-агентные системы больших языковых моделей

Идея использования нескольких автономных программных агентов для совместного решения задач восходит к классическим работам по распределенному искусственному интеллекту. С появлением больших языковых моделей (от англ. Large Language Models, LLM) концепция мульти-агентных систем (от англ. Multi-Agent System, MAS) переживает второе рождение — роль агента теперь играет языковая модель с заданной системной подсказкой и доступным набором инструментов. Обзорная работа [1] систематизирует быстрорастущую совокупность исследований и выделяет несколько устойчивых архитектурных образцов.

Первой широко известной системой подобного класса является фреймворк AutoGen [4], в котором несколько агентов обмениваются сообщениями в гибком формате многораундового диалога и могут вызывать внешние инструменты, в том числе подключать человека. AutoGen не фиксирует топологию заранее — структура взаимодействия определяется императивным кодом конкретного сценария. Это позволяет реализовать любую конфигурацию, но лишает фреймворк возможности сравнивать топологии между собой в рамках единой архитектуры.

Иной образец продемонстрирован в системе ChatDev [8], где процесс разработки программного обеспечения моделируется последовательностью ролевых агентов — генерального директора, программиста, тестировщика — работающих по принципу водопадной модели. Метод MetaGPT [31] развивает идею за счет введения стандартных операционных процедур, что позволяет снизить число ошибок согласования между ролями. Эти работы демонстрируют, что разделение труда между специализированными агентами повышает качество решения сложных задач, однако топология взаимодействия в них жестко зашита в архитектуру.

Параллельно развивается направление, в котором отдельный агент итеративно улучшает собственный ответ. Метод Self-Refine [41] реализует цикл генерация — критика — переработка, а метод Reflexion [38] расширяет цикл вербальным подкреплением — модель формирует словесную обратную связь и хранит ее в эпизодической памяти. Несмотря на привлекательную простоту, эти методы по существу остаются одноагентными и не используют разнообразие точек зрения, доступное мульти-агентным конфигурациям.

1.2 Коммуникационные топологии агентов

Под коммуникационной топологией понимается граф, описывающий, какие агенты могут обмениваться сообщениями и в каком порядке. В существующих фреймворках топология выбирается архитектурно и не пересматривается во время выполнения задачи. Тем не менее ряд недавних работ ставит вопрос об оптимальной структуре такого графа. Используемая в настоящей

работе таксономия графовых структур включает пять базовых типов — звезда (центральный координатор и периферийные работники), цепочка (линейный конвейер), полносвязная (каждый агент общается с каждым), дебатная (два оппонента и арбитр) и иерархическая (двух- или многоуровневое разделение труда). Указанные пять типов и составляют пространство сравнения, исследуемое в настоящей работе. Схематическое изображение пяти базовых топологий приведено на рисунке 1.1.

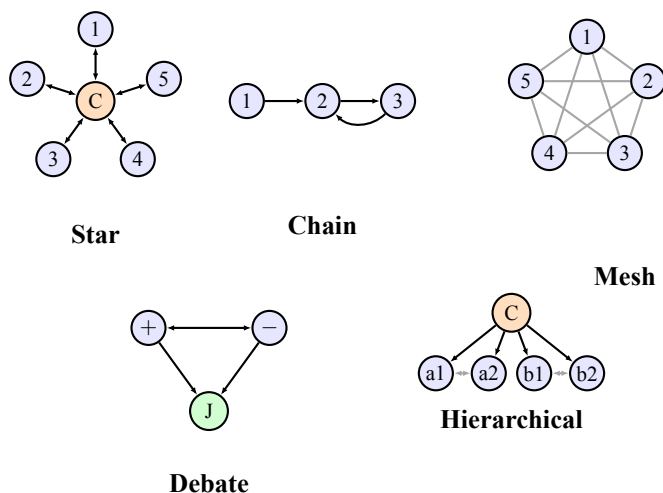


Рисунок 1.1. Схематическое изображение пяти базовых коммуникационных топологий, рассматриваемых в работе. Узлы — агенты; С — координатор, J — арбитр, + и — — оппоненты в дебатах, a1, a2, b1, b2 — участники подкоманд. Сплошные направленные ребра обозначают передачу управления, серые двунаправленные — обмен сообщениями в общей шине.

Работа GPTSwarm [17] рассматривает мульти-агентную систему как обучаемый граф вычислений и оптимизирует его параметры на корпусе задач. Авторы показывают, что оптимизированные графы превосходят случайные конфигурации, но в качестве пространства поиска используют только статичные структуры. Метод G-Designer [16] применяет графовые нейронные сети (от англ. Graph Neural Network — GNN) для автоматического подбора топологии под конкретную задачу. Несмотря на адаптацию к типу задачи, выбор топологии производится однократно и не пересматривается в ходе решения.

В работе [40] систематически исследуются закономерности масштабирования мульти-агентных систем при увеличении числа агентов. Авторы обнаруживают, что топологии типа малого мира (small-world) демонстрируют

повышенную робастность к ошибкам отдельных агентов, однако сравнение ведется между статичными конфигурациями и не учитывает динамику решения задачи. Метод DyLAN [28] предлагает динамический выбор состава команды агентов на каждом шаге, но варьирует только участников — сама структура связей между ними остается фиксированной. Метод Mixture-of-Agents [32] организует агентов в многослойную ансамблевую конфигурацию с агрегацией ответов, которая также не меняется во время решения.

Промежуточные между однозвенными и мульти-агентными решения представлены работами Tree-of-Thoughts [44] и CAMEL [6]. Первая разворачивает дерево вариантов рассуждения с поиском в ширину, вторая моделирует ролевой диалог двух агентов. В обоих случаях структура взаимодействия определяется заранее и фиксируется на протяжении всей задачи. Ближе к проблеме адаптации структуры подходят работы по дебатам между языковыми моделями [21], где несколько моделей обмениваются аргументами для повышения качества вывода, однако число оппонентов и протокол обмена в них зафиксированы заранее.

Общая характеристика рассмотренных работ заключается в том, что выбор топологии в них производится либо архитектурно (на этапе проектирования системы), либо заранее (на этапе получения новой задачи). Возможность переключения топологии в процессе решения, в зависимости от фазы и наблюдаемых сигналов, в этих работах не рассматривается.

1.3 Механизмы адаптации топологий

Адаптация мульти-агентных систем во время выполнения исследуется существенно реже. В проанализированной выборке преобладают два направления адаптации, ни одно из которых не покрывает интересующий нас случай динамического переключения топологии между фазами решения.

Первое направление — адаптация состава команды. Уже упоминавшийся метод DyLAN [28] выбирает на каждом шаге подмножество агентов, наиболее подходящее для текущего контекста. Аналогично работа AgentVerse [3] использует механизм найма агентов, при котором система формирует коман-

ду под задачу из заранее заданного пула. В обоих случаях изменяется не структура взаимодействия, а множество участников — сама схема обмена сообщениями остается фиксированной.

Второе направление — оптимизация конфигурации перед запуском задачи. Системы GPTSwarm [17] и G-Designer [16] принимают решение о структуре графа агентов до начала вычислений и не пересматривают его в дальнейшем. Такие методы можно рассматривать как однократную офлайн-адаптацию, которая полезна для классов однородных задач, но недостаточна для задач со сложной фазовой структурой.

Сложные задачи неоднородны по своей внутренней структуре. Фаза планирования может требовать дебатного формата для генерации альтернатив, фаза исполнения — иерархического разделения труда, фаза проверки — независимой рецензии. Отсюда следует естественная гипотеза — статическая топология, оптимальная для усредненной картины, может быть субоптимальной для каждой отдельной фазы. Эта гипотеза мотивирует разработку механизма переключения топологии в процессе выполнения задачи, который и является одним из центральных предметов настоящего исследования.

1.4 Безопасность и устойчивость топологий

Отдельное направление работ посвящено анализу уязвимостей мульти-агентных систем и зависимости этих уязвимостей от топологии. Работа NetSafe [35] систематически исследует устойчивость различных конфигураций к воздействию скомпрометированного агента. Авторы устанавливают, что плотные топологии (полносвязная, звезда) более подвержены каскадному распространению ошибок, тогда как разреженные структуры (цепочка) ограничивают радиус поражения. Работа ТОМА [42] развивает анализ на случай многошаговых атак, в которых злоумышленник может последовательно воздействовать на несколько агентов с учетом их связности.

Эти результаты важны для нашего исследования по двум причинам. Во-первых, они показывают, что выбор топологии не сводится к чистому критерию качества решения — надежность системы к ошибкам отдельных компо-

нентов также является функцией структуры графа. Во-вторых, они мотивируют включение в систему защитных правил переключения топологий, которые предотвращают патологические режимы взаимодействия (в частности, частое осцилляционное переключение между топологиями без существенного прогресса в решении). Реализация таких правил в настоящей работе обсуждается в главе 3.

1.5 Человек в контуре управления

Подключение человека к работе автоматизированных систем (от англ. Human-in-the-Loop, HITL) — давно установленная практика. Обзорная работа [34] систематизирует методы интеграции человека в контур машинного обучения, выделяя такие функции человека как разметка данных, исправление предсказаний, активный отбор примеров. Классическая работа Хорвица [20] ввела принципы смешанной инициативы, при которой автоматизированная система и человек поочередно ведут процесс решения задачи в зависимости от уверенности каждой стороны. Эти принципы остаются актуальными для проектирования современных мульти-агентных систем.

Отдельную линию исследований представляют методы обучения языковых моделей с обратной связью от человека. Работа Кристиано и соавторов [10] предложила обучение с подкреплением от человеческих предпочтений, что позднее легло в основу метода InstructGPT [36] и стало стандартной техникой выравнивания моделей. В этих работах человек выступает в роли поставщика сигнала вознаграждения. Применительно к мульти-агентным системам вопрос ставится принципиально иначе — человек участвует не только в обучении, но и в каждом отдельном вызове системы, что требует формализации его роли в графе обмена сообщениями.

Обзор [1] перечисляет несколько типичных паттернов включения человека — координатор, рецензент, судья при споре агентов, — но не предлагает экспериментального сравнения их эффективности. Фреймворк AutoGen [4] допускает подключение человека как агента в диалоге, однако роль человека задается архитектурно и не варьируется в ходе экспериментов. Отдельные

работы исследуют человека как судью в задачах оценки качества генерации с использованием шкалы NASA-TLX (от англ. NASA Task Load Index, индекс когнитивной нагрузки NASA) [18], но эта функция выделена в самостоятельную ветвь литературы и не пересекается с проектированием рабочего графа агентов.

Современные оркестраторы мульти-агентных систем, такие как Magentic-One [29], реализуют разделение между ведущим агентом-координатором и подчиненными работниками, при этом человек явно подключен как опциональный участник в роли постановщика задачи. Однако сравнения эффективности разных ролей человека в этой работе также не проводится.

В проанализированной выборке работ за 2022–2026 годы систематических исследований того, какая роль человека и в какой фазе наиболее эффективна для разных типов задач, не обнаружено. Эта лакуна составляет второй центральный предмет настоящего исследования.

1.6 Сравнительная аналитическая таблица методов

Рассмотренные методы естественно сопоставить по нескольким критериям, релевантным для исследовательских вопросов работы. Соответствующее сравнение приведено в таблице 1.1. Знак «+» означает наличие свойства, «–» — его отсутствие, «±» — частичную реализацию.

1.7 Что отсутствует в существующих работах

Из таблицы 1.1 видно, что три столбца — систематическое сравнение топологий на едином стенде, адаптация топологии во время решения задачи, явная роль человека в структуре графа — в существующих работах представлены лишь эпизодически и в проанализированной выборке не встречаются одновременно. Метод G-Designer [16] систематически сравнивает топологии, но не адаптирует их во время выполнения задачи. AutoGen [4] позволяет подключать человека, но не формализует его роль и не сравнивает разные конфигурации. DyLAN [28] адаптирует состав, но не структуру связей. Ни один из методов не объединяет в одном эксперименте сравнение нескольких топо-

Таблица 1.1. Сравнение существующих методов по ключевым критериям

Метод	Год	MAS	Сравн. топ.	Адапт. состава	Адапт. топ. runtime	HITL	Безопасн.
ReAct [37]	2022	—	—	—	—	—	—
CAMEL [6]	2023	+	—	—	—	—	—
AutoGen [4]	2023	+	—	—	—	+	—
Tree-of-Thoughts [44]	2023	—	—	—	—	—	—
Reflexion [38]	2023	—	—	—	—	—	—
LLM debate [21]	2023	+	—	—	—	—	—
DyLAN [28]	2023	+	—	+	—	—	—
AgentVerse [3]	2023	+	—	+	—	—	—
ChatDev [8]	2024	+	—	—	—	—	—
MetaGPT [31]	2024	+	—	—	—	—	—
GPTSwarm [17]	2024	+	±	—	—	—	—
G-Designer [16]	2024	+	+	—	—	—	—
Self-Refine [41]	2024	—	—	—	—	—	—
MoA [32]	2024	+	—	—	—	—	—
Scaling MAS [40]	2024	+	±	—	—	—	±
Magentic-One [29]	2024	+	—	—	—	±	—
NetSafe [35]	2024	+	+	—	—	—	+
TOMA [42]	2024	+	±	—	—	—	+
Настоящая работа	2026	+	+	—	+	+	±

Обозначения колонок — **MAS** мульти-агентная архитектура, **Сравн. топ.** систематическое сравнение нескольких топологий на едином стенде, **Адапт. состава** динамическое изменение участников, **Адапт. топ. runtime** переключение структуры графа в процессе решения задачи, **HITL** интеграция человека в контур управления как полноценного участника графа, **Безопасн.** учет устойчивости к скомпрометированному агенту и каскадным сбоям. Знак «+» означает наличие свойства, «—» — его отсутствие, «±» — частичную реализацию.

логий, механизм их переключения во время решения и многомерное варьирование роли человека.

Эта совокупность отсутствующих свойств и составляет исследовательскую лакуну, заполнение которой является целью настоящей работы. В частности, настоящая работа предлагает — во-первых, единый программный стенд, на котором сравниваются пять различных топологий на едином корпусе из четырех классов задач; во-вторых, мета-топологию, переключающую активную базовую топологию в зависимости от фазы решения; в-третьих, формализацию пяти ролей человека и экспериментальную оценку их влияния на каждую топологию по полному факторному плану.

1.8 Выводы по главе

Проведенный аналитический обзор позволяет сделать несколько содержательных выводов. Мульти-агентные системы больших языковых моделей

являются активно развивающейся областью с устоявшимся набором архитектурных образцов, однако коммуникационная топология в них обычно фиксируется на этапе проектирования и не пересматривается во время выполнения задачи. Существующие механизмы адаптации ограничены либо составом команды, либо однократным офлайн-выбором структуры графа, что недостаточно для задач со сложной фазовой структурой. Безопасность мульти-агентных систем существенно зависит от топологии, что подтверждает необходимость защитных правил при ее динамическом изменении. Систематического исследования роли человека в графе агентов в проанализированной выборке работ не обнаружено.

На пересечении трех направлений — сравнение топологий, их динамическая адаптация, формализованная роль человека — лежит лакуна, заполнение которой ставится в качестве научной цели настоящей работы. Эта цель формализуется в виде четырех исследовательских вопросов в разделе 2.

2 Постановка задачи и методология исследования

Настоящая глава формализует объект исследования, фиксирует исследовательские вопросы и описывает методологию их эмпирической проверки. Сначала вводится математическая модель мульти-агентной системы, затем определяются шесть рассматриваемых коммуникационных топологий и пять ролей человека в контуре управления. Далее формулируется система оценочных метрик, обосновывается выбор корпуса задач и моделей, и описывается воронка из четырех экспериментов, последовательно отвечающих на поставленные исследовательские вопросы.

2.1 Формальная модель мульти-агентной системы

Мульти-агентная система рассматривается как кортеж

$$\mathcal{M} = \langle V, E, R, \mathcal{T}, F, h \rangle, \quad (2.1)$$

где $V = \{v_1, \dots, v_n\}$ — конечное множество агентов, каждый из которых реализован как языковая модель с фиксированной системной подсказкой и набором инструментов; $E \subseteq V \times V$ — множество допустимых направленных ребер коммуникации, образующее коммуникационный граф; $R: V \rightarrow \mathcal{R}$ — отображение агентов в множество ролей $\mathcal{R}$ (планировщик, исследователь, исполнитель, критик, дебатер, координатор); $\mathcal{T}$ — текущая задача с целевой функцией оценки качества решения; F — конечное множество фаз решения задачи, представленных автоматом $\{planning, execution, verification, done\}$; h — (опционально) участник-человек с одной из пяти заданных ролей.

В классических работах граф E определяется архитектурно и не меняется в ходе выполнения задачи. В настоящем исследовании вводится расширение модели — структура E становится функцией текущей фазы $f \in F$ и

наблюдаемых сигналов σ агентов:

$$E = E(f, \sigma, t), \quad (2.2)$$

где t — индекс итерации. Возможность такого переключения и составляет суть исследуемой адаптивной мета-топологии.

Множество ролей фиксировано и состоит из шести значений — $\mathcal{R} = \{planner, researcher, executor, critic, debater, coordinator\}$. Множество ролей человека фиксировано и содержит пять значений, формализованных в разделе 2.3. Фазы F упорядочены отношением $planning \prec execution \prec verification \prec done$ с монотонной семантикой переходов. Сигналы σ агентов представляют собой вектор булевых индикаторов фиксированной длины, в который входят флаги завершения этапа, признаки тупикового состояния, счетчики отказов критика и другие наблюдаемые признаки прогресса.

Задача оптимизации мульти-агентной системы формулируется как многокритериальная и состоит в нахождении конфигурации $\langle E, R, h \rangle$, доминирующей конкурирующие конфигурации одновременно по трем осям — качеству решения $q(\mathcal{M})$, суммарной стоимости вызовов $c(\mathcal{M})$ и времени выполнения $\tau(\mathcal{M})$. Поскольку чистый Парето-оптимум обычно не единственен, в работе используется скаляризация с фиксированными порогами улучшения. Конфигурация $\mathcal{M}_1$ считается выигрышной относительно $\mathcal{M}_0$, если выполняется одно из условий

$$\frac{q(\mathcal{M}_1) - q(\mathcal{M}_0)}{q(\mathcal{M}_0)} \geq 0,05 \quad \text{или} \quad \frac{c(\mathcal{M}_0) - c(\mathcal{M}_1)}{c(\mathcal{M}_0)} \geq 0,15 \quad \text{при} \quad q(\mathcal{M}_1) \geq q(\mathcal{M}_0). \quad (2.3)$$

Пороги 5% для качества и 15% для стоимости зафиксированы до проведения основных прогонов и обоснованы в разделе 2.4.

2.2 Шесть рассматриваемых топологий

Пространство сравнения составляют шесть топологий. Пять из них являются статичными и реализуют пять базовых классов коммуникационных

структур, перечисленных в обзоре литературы. Шестая является мета-топологией, переключающей активную статическую топологию в зависимости от фазы.

Star — центральный координатор v_c последовательно обращается к специализированным работникам $v_1, \dots, v_k$ и агрегирует их ответы. Граф ребер $E_{\text{star}} = \{(v_c, v_i), (v_i, v_c) \mid i = 1, \dots, k\}$.

Chain — линейный конвейер из трех агентов $v_p \rightarrow v_e \rightarrow v_c$ (планировщик, исполнитель, критик) с условным циклом доработки. Граф направленный ациклический с обратной связью от критика к исполнителю.

Mesh — полносвязная топология с общей шиной сообщений. Граф $E_{\text{mesh}} = (V \times V) \setminus \{(v, v)\}$, обмен организован по принципу циклической очереди, решение принимается голосованием.

Debate — пара оппонентов v_+ и v_- генерирует параллельные альтернативные решения, после чего судья v_j выбирает победителя или синтезирует ответ. Граф двудольный с агрегатором.

Hierarchical — двухуровневая иерархия из координатора верхнего уровня v_c^{top} и двух подкоманд $T_a, T_b \subset V$, каждая из которых сама по себе является графом. Подкоманды работают параллельно, координатор синтезирует итог.

Adaptive — мета-топология, активирующая на каждой итерации одну из пяти базовых конфигураций. Решение о выборе принимается маршрутизатором согласно формуле (2.2). Псевдокод алгоритма приведен в таблице 2.1, общая концептуальная схема взаимодействия блоков системы изображена на рисунке 3.1 главы 3, архитектура реализации описана в разделе 3.3.

2.3 Пять ролей человека в контуре управления

Пять ролей человека формализованы по аналогии с типовыми позициями в человеческих организациях. **Координатор** (ρ_C) — управляет планом и делегированием, активен в иерархических и полносвязных топологиях. **Рецензент** (ρ_R) — проверяет промежуточные результаты и инициирует доработку, естественно вписывается в Star и Chain. **Судья** (ρ_J) — выносит финальный вердикт при наличии альтернатив, релевантен для Debate. **Равноправный участник** (ρ_P) — вносит альтернативные мнения наравне с аген-

Таблица 2.1. Алгоритм работы адаптивной мета-топологии

Шаг	Действие
1	Инициализировать фазу $f \leftarrow planning$, начальную топологию $K \leftarrow K_0$, состояние $s \leftarrow s_0$ и журнал $\mathcal{J} \leftarrow \emptyset$.
2	Пока $f \neq done$ и $c(s) < B$ выполнять шаги 3–10.
3	Применить маршрутизатор фаз $f' \leftarrow PhaseRouter(s, f)$.
4	Применить маршрутизатор топологий $K' \leftarrow TopologyRouter(s, K, f')$.
5	Если $Guard(s, K, K') = block$, то $K' \leftarrow K$ и записать $guard_override$ в $\mathcal{J}$.
6	Если $K' \neq K$, применить ворота перехода состояния $s \leftarrow TransitionGate(s, K, K')$ и записать переход в $\mathcal{J}$.
7	Исполнить подграф выбранной топологии $s \leftarrow Subgraph_{K'}(s)$.
8	Обновить $K \leftarrow K'$ и $f \leftarrow f'$.
9	Записать журнал итерации в $\mathcal{J}$.
10	Перейти к шагу 2.
11	Вернуть результат $y(s)$ и журнал $\mathcal{J}$.

тами, вписывается в Mesh. **Наблюдатель** (ρ_M) — следит за процессом и вмешивается только в критических случаях (превышение бюджета, зависание, циклы), применим в любой топологии.

Не все пары (топология, роль) являются осмысленными. Рассматриваются пять статических топологий и пять ролей человека — полное произведение содержит 25 пар. Из него исключаются три заведомо вырожденные комбинации. Пара (Debate, Coordinator) исключается, поскольку координатор в дебатной топологии эквивалентен судье. Пары (Chain, Peer) и (Hierarchical, Peer) исключаются, поскольку равноправный участник ломает линейную или иерархическую структуру. После исключений пространство содержит $5 \times 5 - 3 = 22$ осмысленные комбинации топология $\times$ роль.

2.4 Система оценочных метрик

Метрики разделены на три группы. Первая группа — базовые метрики качества и эффективности — применима ко всем экспериментам и ко всем топологиям.

- $q \in [0, 1]$ — агрегированный показатель качества решения. Конкретная формула зависит от класса задач — для задач программирования это метрика $pass@1$ (доля задач, для которых сгенерированный код проходит все предзаданные модульные тесты с первой попытки); для ма-

тематических задач — exact match (точное совпадение числового ответа с эталоном); для творческой генерации — полусумма метрики ROUGE-L (от англ. Recall-Oriented Understudy for Gisting Evaluation, Longest common subsequence variant) и доли покрытых концептов; для анализа данных — exact match числового ответа.

- c — суммарная стоимость вызовов языковых моделей в долларах США за один прогон.
- τ — время выполнения прогона по настенным часам (от англ. wall-clock) в секундах.
- r_{fail} — доля прогонов, завершившихся со статусом, отличным от успешно выполненного.
- N_{iter} — число итераций основного цикла.

Вторая группа — метрики, специфичные для адаптивной мета-топологии.

- N_{switch} — число фактических переключений активной топологии за один прогон.
- r_{guard} — доля решений маршрутизатора, заблокированных защитными правилами.
- γ — доля бюджета, потраченная маршрутизатором на собственные вызовы языковой модели; критичная метрика, отвечающая на вопрос, не превышает ли стоимость адаптации получаемый от нее выигрыш.
- r_{hurt} — доля задач, на которых адаптивная конфигурация дает качество хуже лучшей статической конфигурации. Является ключевым индикатором безопасности механизма адаптации.
- Δ_{oracle} — разрыв качества между оракул-выбором лучшей статической топологии для каждого корпуса и фактическим маршрутизатором, формально определяемый по формуле (2.4). Оракул реализован как `reg-task best-of` — для каждого корпуса задач выбирается статическая топология с максимальным средним качеством на этом корпусе по результатам базового эксперимента E1. Метрика служит верхней границей достижимого качества для класса задач при заданном пространстве статических топологий.

$$\Delta_{\text{oracle}} = q_{\text{oracle}} - q_{\text{router}}, \quad \text{где} \quad q_{\text{oracle}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} q(K_c^*, c), \quad (2.4)$$

где $\mathcal{C}$ — множество корпусов задач, $K_c^* = \arg \max_K q(K, c)$ — оптимальная статическая топология для корпуса c , $q(K, c)$ — среднее качество топологии K на корпусе c по базовому эксперименту.

Третья группа — метрики, относящиеся к взаимодействию с человеком.

- N_{int} — число обращений системы к человеку за один прогон.
- L_{cog} — прокси-метрика когнитивной нагрузки для запусков с симулятором человека, формально определяемая по формуле (2.5). В экспериментах с реальными участниками заменяется на стандартную шкалу NASA-TLX.

$$L_{\text{cog}} = w_n \cdot \tilde{N}_{\text{int}} + w_l \cdot \tilde{L}_{\text{ctx}} + w_\tau \cdot \tilde{\tau}_{\text{resp}}, \quad w_n = w_l = w_\tau = \frac{1}{3}, \quad (2.5)$$

где $\tilde{N}_{\text{int}}$, $\tilde{L}_{\text{ctx}}$ и $\tilde{\tau}_{\text{resp}}$ — нормированные на отрезок $[0, 1]$ значения числа обращений к участнику, средней длины предоставляемого ему контекста и средней задержки ответа симулятора соответственно. Нормировка выполняется по квантилям распределения наблюдаемых значений за все прогоны эксперимента. Равные веса выбраны априорно до начала прогонов как нейтральная свертка трех независимых компонентов нагрузки.

Адаптивная конфигурация считается выигрышной относительно наилучшей статической, если она дает прирост качества не менее 5% либо снижение стоимости не менее 15% при сохранении качества. Пороги фиксируются предварительно, до проведения основных прогонов, что соответствует практике предварительной регистрации гипотез в эмпирических исследованиях.

2.5 Корпус задач и обоснование его выбора

Эксперименты проводятся на стратифицированной выборке из четырех открытых корпусов, покрывающих четыре качественно различных класса за-

дач — программирование, математические рассуждения, творческая генерация и анализ данных. Сравнение корпуса с альтернативами приведено в таблице 2.2.

Таблица 2.2. Сравнение рассмотренных корпусов задач

Корпус	Класс	Размер	Метрика	Решение по включению
HumanEval [15]	Программирование	164	pass@1	Включен; стандарт сравнения.
HumanEval+ [23]	Программирование	164	pass@1+	Не включен; превышает покрытие основной выборки.
LiveCodeBench [27]	Программирование	500+	pass@1	Резерв для проверки на загрязнение.
GSM8K [43]	Математ. рассуждения	8500	exact match	Включен; стандарт оценки.
MATH [30]	Математ. рассуждения	12500	exact match	Не включен; сложность выше возможностей рабочих моделей.
MMLU-Pro [33]	Множеств. рассужд.	12000	accuracy	Не включен; формат ответа не подходит для дебатов топологии.
CommonGen [11]	Творческая генерация	35000	ROUGE-L + cov	Включен; покрывает open-ended генерацию с ограничениями.
InfiAgent-DABench [22]	Анализ данных	257	numeric exact	Включен; единственный корпус с замкнутым ответом по аналитике данных.
HellaSwag [19]	Здравый смысл	10000	accuracy	Не включен; качество современных моделей близко к потолку.

Выбор обусловлен совокупностью требований. Каждый класс задач представлен ровно одним корпусом, чтобы избежать смешения эффектов корпуса и класса. Каждый корпус допускает автоматическую объективную оценку, исключая субъективность оценщика (юнит-тесты, точное совпадение числа, метрики покрытия концептов). Размер задач в каждом корпусе позволяет уложиться в один прогон рабочих моделей. От рассматриваемых для каждого класса альтернатив выбранные корпуса отличаются распространенностью в литературе и зрелостью методики оценки, что обеспечивает сопоставимость результатов с предшествующими работами. Из каждого корпуса производится стратифицированная выборка по 15 задач, итого 60 задач в одном базовом запуске. Каждая задача повторяется с тремя различными зернами генерации (для контроля стохастичности выборки токенов), что в комби-

рации с пятью статическими топологиями дает $60 \times 3 \times 5 = 900$ прогонов в эксперименте E1.

2.6 Языковые модели и инфраструктура

Распределение моделей по ролям спроектировано таким образом, чтобы изолировать переменную «модель» от переменных «топология» и «роль человека». Все рабочие агенты (планировщик, исследователь, исполнитель, критик, дебатер), а также модели маршрутизаторов и симулятора человека используют одну и ту же модель Cerebras gpt-oss-120b, обеспечивающую низкую стоимость вызовов и высокую скорость генерации (порядка трех тысяч токенов в секунду на инфраструктуре Cerebras WSE). Иерархия «работник слабее критика» реализуется не разной моделью, а различиями в системных подсказках и в параметре строгости рассуждения. Это решение принципиально для чистоты эксперимента, поскольку фиксирует одну и ту же модельную мощность для всех сравниваемых конфигураций.

Судья оценки качества решений выделен в отдельную линию моделей — OpenAI gpt-4.1-mini с тремя независимыми вызовами по схеме самосогласованности и нулевой температурой выборки. Использование модели из другой семейства весов является ключевым методологическим выбором — оно исключает риск того, что система оценивает собственные ответы и формирует петлю положительной обратной связи. Подтверждающие прогоны (см. ниже) проводятся на двух кросс-семейных моделях — Cerebras zai-glm-4.7 и OpenAI gpt-4o, — что позволяет разделить эффекты адаптивной топологии от эффектов конкретного семейства весов.

Выбор основных моделей опирался на четыре критерия. Во-первых, доступность отказоустойчивого программного интерфейса с поддержкой вызова инструментов и структурированного вывода — требование, обеспечивающее воспроизводимое исполнение многошаговых циклов агентов. Во-вторых, низкая стоимость единичного вызова, позволяющая выполнить полную сетку из нескольких тысяч прогонов в рамках бюджета бакалаврской работы. В-третьих, высокая пропускная способность, исключаящая инфраструктур-

ные ограничения как причину различий между топологиями. В-четвертых, доступность из Российской Федерации без юридических и финансовых препятствий. Под эти критерии попадают Cerebras gpt-oss-120b (основная инфраструктура), OpenAI gpt-4.1-mini (судья оценки) и две подтверждающие модели — Cerebras zai-glm-4.7 как представитель другого семейства весов на той же инфраструктуре и OpenAI gpt-4o как frontier-модель другого провайдера. Подробный сравнительный анализ альтернативных моделей и поставщиков приведен в техническом отчете проекта.

Инфраструктура экспериментов описана в главе 3 и включает оркестрацию на LangGraph, хранение метаданных в PostgreSQL версии 16, объемных журналов — в формате Apache Parquet. Параллелизм выполнения настроен на двенадцать процессов одновременно с асинхронной обработкой запросов внутри каждого процесса.

2.7 Дизайн экспериментов — воронка

Прямой эксперимент по всему декартову произведению переменных (шесть топологий, пять ролей человека, три режима маршрутизации, четыре класса задач) дал бы более десяти тысяч ячеек и потребовал бы непропорционального бюджета. Вместо этого исследование организовано как последовательность из четырех экспериментов, каждый из которых сужает пространство конфигураций по результатам предыдущего. Структура воронки приведена в таблице 2.3.

Таблица 2.3. Структура экспериментальной воронки

Эксп.	Иссл. вопрос	Прогоны	Цель
E1	RQ1	900	Pareto-фронт пяти статических топологий на четырех классах задач.
E2	RQ3	2160	Влияние пяти ролей человека на каждую топологию.
E3	RQ2	720	Адаптивная мета-топология против лучшей статической конфигурации в трех режимах маршрутизации.
E4	RQ4	540	Адаптивная роль человека в сочетании с адаптивной топологией.
Conf.	—	1100	Кросс-семейная валидация выводов на двух подтверждающих моделях.

Исследовательские вопросы формулируются следующим образом.

RQ1. Различаются ли пять статических топологий по количественным показателям качества, стоимости и времени для разных классов задач?

RQ2. Дает ли адаптивная мета-топология значимый прирост качества или экономию стоимости по сравнению с лучшей статической конфигурацией?

RQ3. Какая из пяти ролей человека наиболее эффективна для каждой топологии и для каждого класса задач?

RQ4. Превосходит ли совместная адаптация топологии и роли человека по фазам решения статичные комбинации?

Статистическая обработка включает двухфакторный дисперсионный анализ (two-way ANOVA) с эффектом топологии и эффектом роли в качестве факторов на уровне значимости $\alpha = 0,05$. При нарушении допущений нормальности или равенства дисперсий применяется непараметрический аналог по критерию Краскела–Уоллиса. Парные сравнения средних выполняются с поправкой Бенджамини–Хохберга на множественное сравнение при уровне ложных открытий $FDR = 0,05$. Оценка практической значимости различий проводится через коэффициент размера эффекта Коэна d . Доверительные интервалы рассчитываются по методу непараметрического бутстрапа с тысячей ресэмплов на уровне 95 %.

2.8 Разграничение заимствованных и авторских элементов

Поскольку методология настоящей работы сочетает несколько существующих элементов и предлагает несколько новых, ниже явно фиксируется граница между ними. К заимствованным относятся пять статических топологий (звезда, цепочка, полносвязная, дебатная, иерархическая) как известные паттерны организации мульти-агентного взаимодействия, четыре открытых корпуса задач, языковые модели и стандартный статистический инструментарий (дисперсионный анализ, метод бутстрапа, коэффициент эффекта Коэна). К авторским относятся следующие элементы. Во-первых, адаптивная мета-топология вместе с алгоритмом ее работы, сформулированная в разде-

ле 2.2 и формализованная уравнением (2.2). Во-вторых, формализация пяти ролей человека и соответствующая матрица совместимости топологий с ролями. В-третьих, набор адаптивно- специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{oracle}), формализованных в разделе 2.4. В-четвертых, вороночная структура эмпирического исследования с предварительной фиксацией порогов из формулы (2.3) и кросс-семейным подтверждением на двух дополнительных моделях.

2.9 Выводы по главе

В главе формализована мульти-агентная система как кортеж из шести компонентов с возможностью динамического переключения структуры коммуникационного графа в зависимости от фазы решения. Определены шесть рассматриваемых топологий, пять ролей человека в контуре управления и три группы оценочных метрик, включая специфичные для адаптивной метатопологии характеристики — число переключений, долю заблокированных решений маршрутизатора, стоимостный вклад самого маршрутизатора, долю задач с регрессией качества и разрыв до оракула. Обоснован выбор корпуса задач из четырех открытых бенчмарков и линии языковых моделей с кросс-семейным судьей. Сформулированы четыре исследовательских вопроса и описана воронка из четырех основных экспериментов с подтверждающим прогоном на дополнительных семействах моделей. Граница между заимствованными и предложенными в работе элементами зафиксирована в разделе 2.8 и развернута в самостоятельном разделе авторского вклада (5.5). Изложенный план непосредственно реализуется в инфраструктуре главы 3 и эмпирически проверяется в главе 4.

3 Архитектура программной системы

Программная инфраструктура, обеспечивающая выполнение всех описанных в главе 2 экспериментов, реализована в виде самостоятельного исследовательского фреймворка с условным именем АТМ (от англ. Adaptive Topologies for Multi-agent systems — адаптивные топологии для мульти-агентных систем). Фреймворк написан на языке Python версии не ниже 3.11 и распространяется как пакет `atm`. Настоящая глава описывает архитектурное устройство системы с акцентом на те решения, которые имеют непосредственное отношение к научным гипотезам работы, — адаптивному переключению топологий и интеграции человека в контур управления. Полный исходный код размещен в открытом репозитории, ссылка приведена в приложении Г.

Общая функциональная схема системы изображена на рисунке 3.1. На вход подается описание задачи из выбранного бенчмарка и конфигурация эксперимента в формате YAML (от англ. YAML Ain't Markup Language — человекочитаемый формат сериализации). На выход поступают журналы взаимодействия агентов, агрегированные метрики качества и стоимости, а также структурированные записи о фазах решения, переключениях топологий и взаимодействиях с человеком. Внутри между входом и выходом последовательно работают пять архитектурных блоков — оркестратор эксперимента, менеджер фаз и маршрутизаторы, активная топология с агентами, шлюз взаимодействия с человеком и подсистема наблюдаемости.

3.1 Обзор архитектуры и слои

Кодовая база организована в виде тринадцати функционально изолированных слоев (пакетов высокого уровня), каждый из которых отвечает за один аспект системы и содержит несколько программных модулей. Перечень слоев и их назначений приведен в таблице 3.1. Такая декомпозиция позволяет независимо тестировать компоненты, заменять реализации (например, провайдера языковой модели или шлюз человека в контуре управления от англ. Human-in-the-Loop, HITL) и добавлять новые топологии без изменения су-

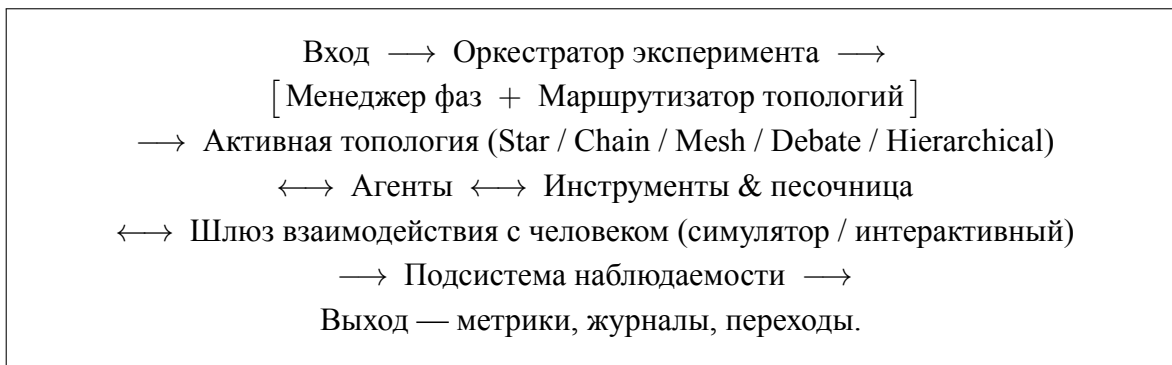


Рисунок 3.1. Функциональная схема предлагаемой мульти-агентной системы. Прямоугольниками выделены архитектурные блоки фреймворка, двунаправленные стрелки изображают многократные обмены сообщениями в пределах одной итерации. Развернутая схема с указанием слоев агентов, внешних служб и подсистемы хранения приведена в приложении А.

ществующего кода.

Таблица 3.1. Функциональные слои фреймворка atm

Слой	Назначение
core	Базовые типы и состояние графа — сообщения, фазы, роли, контейнеры состояния AgentState, SharedState, GraphState.
llm	Унифицированная оболочка над провайдерами больших языковых моделей, подсчет стоимости, бюджетные ограничители, детерминированный FakeLLM для тестов.
tools	Реестр инструментов и песочница на основе среды контейнеризации Docker для исполнения генерируемого кода.
agents	Реализация пяти ролей агентов и базовый класс Agent с управляющим циклом вызова инструментов.
topology	Шесть реализаций топологий поверх LangGraph и общий протокол Topology.
phases	Конечный автомат фаз, маршрутизатор топологий и защитные правила переключения.
human	Протокол HumanGateway, симулированный и интерактивный шлюзы, маршрутизатор ролей человека.
storage	Объектно-реляционное отображение (от англ. Object-Relational Mapping, ORM), асинхронные сессии PostgreSQL, запись в формат Apache Parquet.
observability tasks	Перехват событий LangGraph, сериализация и отправка их в хранилище. Реализация и загрузчики бенчмарков HumanEval, GSM8K, CommonGen, InfiAgent-DABench.
evaluation	Судьи на основе языковой модели, оракулы достоверности, агрегатор шкалы NASA-TLX (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA).
experiment	Запуск одиночных и грид-экспериментов, схемы конфигов, командная строка.
analysis	Загрузчики результатов, агрегаторы метрик, построение графиков.

Архитектурный стиль фреймворка опирается на несколько последовательно применяемых паттернов. Во-первых, все расширяемые точки описаны как протоколы языка Python (Topology, HumanGateway, PhaseRouter, TopologyRouter, RoleRouter, Tool) с проверкой соответствия в момент выполнения через декоратор `@runtime_checkable`. Это позволяет подключать сторонние реализации без явного наследования. Во-вторых, для сообщений и переходов используется паттерн редуктора. Состояние графа объединяется при параллельном выполнении агентов через явно зарегистрированные функции слияния, гарантирующие идемпотентность и дедупликацию по идентификатору сообщения. В-третьих, повторяющиеся сущности — топологии, инструменты, оракулы — регистрируются в реестрах с декораторной семантикой.

Технологический стек системы выбран из соображений воспроизводимости и масштабируемости. Оркестрация графов агентов выполнена на библиотеке LangGraph, использующей чекпойнтер для возобновления прерванных запусков. Метаданные экспериментов хранятся в PostgreSQL версии 16 через асинхронные сессии SQLAlchemy 2.x, объемные журналы взаимодействия — в файлах Apache Parquet. Конфигурации экспериментов описаны на YAML и валидируются композитными моделями библиотеки Pydantic поверх OmegaConf. Линтер ruff и статический анализатор типов mypy применяются в строгом режиме.

Часть архитектурных решений заимствована из существующих работ и фреймворков, часть является авторской разработкой. Точное разграничение приведено в таблице 3.2.

3.2 Слой агентов и оболочка над языковой моделью

Базовый класс Agent в модуле `atm.agents` реализует типовой управляющий цикл агента — считывание входящих сообщений, формирование подсказки с учетом скрэтчпада, вызов языковой модели, обработку запрошенных инструментов и публикацию ответа. На основе этого класса определены пять ролей — Planner, Researcher, Executor, Critic, Debater — и дополнительная роль Coordinator, активируемая в иерархических и адаптивных то-

Таблица 3.2. Разделение архитектурных компонентов на заимствованные и авторские

Компонент	Источник	Авторский вклад
Оркестрация графов	LangGraph (внешний)	Адаптация под мета-граф
Пять статичных топологий	Прототипы из обзора литературы	Унифицированная реализация по-верх об-щего прото-тока-ла
Адаптивная мета-топология	Своя разработка	Полностью
Маршрутизаторы фаз	Своя разработка	Полностью, три ре-жима (пра-вило-вый, на ос-нове язы-ковой мо-дели, ора-кул)
Защитные правила переключения	Своя разработка	Полностью, че-тыре типа огра-ничи-телей
Шлюз HITL	Своя разработка	Полностью, три роле-вых марш-рути-затора и поли-тика тайм-аутов

пологиях. Каждая роль определяется YAML-конфигом, содержащим системную подсказку, допустимый набор инструментов, размер скользящего окна сообщений, температуру выборки и параметры управления контекстом.

Скрэтчпад агента (внутренний рабочий контекст) организован по политике скользящего окна с автоматическим сжатием. История сообщений поддерживается в виде скользящего окна фиксированного размера, а при его переполнении старые сообщения сжимаются вспомогательной языковой моделью в краткое резюме. Компромисс между полнотой истории и стоимостью вызовов выбран по результатам пилотного эксперимента, в котором настоящая политика сравнивалась с двумя альтернативами (полная история без сжатия и плоский буфер с жестким отсечением); подробности приведены в разделе 4.

Оболочка `LLMWrapper` в модуле `atm.llm` унифицирует работу с разнородными провайдерами — `OpenAI`, `Anthropic`, `Cerebras` и локальным `FakeLLM`. Класс инкапсулирует подсчет токенов и стоимости вызова на основе таблицы цен `Pricing`, реализует экспоненциальный повтор при временных сбоях провайдера и фиксирует снимок версии модели в первом успешном ответе. Бюджетные ограничители `BudgetGuard` применяются на трех уровнях — одиночный вызов, полный запуск эксперимента, грид-эксперимент в целом, — и при превышении лимита прерывают выполнение с явной диагностикой. Реализация `FakeLLM` поддерживает три режима — сценарный, эхо и воспроизведение по записи, — что обеспечивает детерминированное повторение экспериментов и работоспособность модульных тестов без обращения к внешним сервисам.

3.3 Реализация топологий поверх `LangGraph`

Каждая из шести топологий реализована как самостоятельный класс, удовлетворяющий протоколу `Topology` с двумя обязательными полями — именем (`name`) и фабричным методом (`build`), возвращающим скомпилированный направленный граф состояния `LangGraph`. Граф связывает узлы — агентов и управляющие функции — ребрами трех типов — безусловными,

условными и параллельными через примитив `Send` библиотеки `LangGraph`. Все построенные графы единообразно подключаются к чекпойнтеру и к перехватчику событий слоя `atm.observability`, что обеспечивает однородность журнала вне зависимости от выбранной топологии.

Пять статичных топологий различаются принципом организации коммуникации. В топологии *Star* центральный координатор последовательно вызывает специализированных работников и агрегирует их ответы. Топология *Chain* реализует линейный конвейер с обратными связями — планировщик, исполнитель, критик — и условным циклом доработки. *Mesh* использует общую шину сообщений с диспетчером по принципу циклической очереди и голосованием за финальное решение. *Debate* запускает двух оппонентов параллельно (`debater_pro` и `debater_contra`) с последующим выбором победителя судьей. *Hierarchical* организует двухуровневую иерархию из координатора верхнего уровня и двух подкоманд, каждая из которых представляет собой самостоятельный скомпилированный подграф.

Шестая топология *Adaptive* устроена как мета-граф второго уровня. Каждый ее тик начинается с обращения к маршрутизатору фаз и маршрутизатору топологий, после чего выбранная базовая топология вызывается как подграф. По завершении подграфа ворота переходов (`TransitionGate`) применяют таблицу правил передачи состояния и записывают объект `TopologyTransition` в журнал. Скомпилированные подграфы кэшируются по имени, что исключает повторную компиляцию `LangGraph` при многократных переключениях. Решения маршрутизаторов хранятся в замыкании, а не в общем состоянии, чтобы избежать их перезаписи во время выполнения вложенного подграфа.

3.4 Менеджер фаз и маршрутизатор топологий

Решение задачи в системе моделируется конечным автоматом (от англ. `Finite State Machine` — `FSM`) из четырех состояний — планирование, исполнение, проверка и завершение, — переходы между которыми инициируются сигналами агентов и проверками максимального числа итераций. Слой `atm.phases` содержит две реализации маршрутизатора фаз. Детерминиро-

ванная реализация `RuleBasedPhaseRouter` использует таблицу сигналов и монотонную проверку допустимости перехода — фаза не может сместиться назад. Маршрутизатор на основе языковой модели (`LLMPhaseRouter`) обращается к модели с шаблоном подсказки, парсит результат как структурированный объект формата JSON (от англ. JavaScript Object Notation) и автоматически откатывается на правило при любой ошибке валидации или генерации.

Параллельно с фазами работает маршрутизатор топологий, отвечающий за решение о переключении базовой топологии внутри адаптивной конфигурации. Реализованы три режима. Правильный `RuleBasedTopologyRouter` использует таблицу (фаза $\times$ сигналы) $\rightarrow$ топология, где сигнал `stuck` в фазе исполнения активирует *Mesh*, а сигнал `rejected_count` со значением не ниже трех — *Debate*. Конкретные пороги срабатывания подобраны эмпирически в пилотном эксперименте и обоснованы в разделе 4. `LLMTopologyRouter` принимает решение на основе подсказки с описанием текущего состояния и допустимых топологий, с аналогичным правилowym откатом. `OracleTopologyRouter` читает заранее построенную таблицу решений из файла, что служит верхней границей достижимого качества адаптации.

Защитные правила переключения, реализованные в виде набора замыканий, предотвращают режим частого осцилляционного переключения, при котором система непрерывно меняет активную топологию без существенного прогресса. Применяются четыре ограничения — минимальное число итераций в одной топологии перед сменой (`min_dwell`), интервал между сменами (`cooldown`), максимальное число переключений за фазу и за весь запуск. Если решение маршрутизатора заблокировано хотя бы одним из ограничений, оно фиксируется в журнале со специальной отметкой `guard_override`, что позволяет в дальнейшем оценить долю заблокированных решений как метрику стабильности.

3.5 Шлюз взаимодействия с человеком

Интеграция человека в контур управления абстрагирована единым протоколом `HumanGateway`, единственным методом которого является асинхрон-

ный запрос ответа от человека по описанию контекста (запрос содержит идентификатор запуска и запроса, текущую роль человека, недавнюю историю сообщений и допустимый набор действий). Идемпотентность гарантируется парой идентификаторов (`run_id`, `request_id`). Повторный вызов с тем же идентификатором возвращает ранее полученный ответ без обращения к человеку, что критично для возобновления прерванных запусков после сбоев.

Реализованы два шлюза. `LLMSimulatedGateway` использует языковую модель в роли симулятора человека с системной подсказкой, зависящей от выбранной роли — координатор, рецензент, судья, равноправный участник или наблюдатель. `CLIGateway` ориентирован на интерактивное взаимодействие через стандартный поток ввода-вывода и применяется в экспериментах с реальными участниками. Маршрутизатор ролей человека `RoleRouter` выбирает активную роль динамически — правилочная таблица или решение языковой модели — либо фиксированно (`FixedRoleRouter`). Политика обработки тайм-аутов отвечает за сценарии, в которых человек не успевает ответить в отведенное время. Доступны три варианта — откат к языковой модели, пропуск шага, повтор, — что делает поведение системы предсказуемым в любых условиях.

3.6 Хранение, командная строка и воспроизводимость

Метаданные экспериментов хранятся в семи таблицах PostgreSQL — `experiments`, `runs`, `phases`, `topology_transitions`, `human_interactions`, `budget_events`, `llm_calls`. Схема включает индексы по типичным аналитическим запросам (по эксперименту, по топологии, по статусу) и составной индекс по паре (эксперимент, статус) для быстрой выборки невыполненных запусков. Объемные журналы взаимодействия агентов записываются в файлы Apache Parquet через асинхронный писатель и партиционируются по идентификатору запуска. Миграции схемы базы данных управляются библиотекой Alembic, текущая ревизия фиксирует поддержку воспроизведения запусков.

Командная строка `atm`, реализованная на библиотеке Typer, объединяет

семь команд для запуска и сопровождения экспериментов, описание которых приведено в таблице 3.3.

Таблица 3.3. Команды интерфейса командной строки фреймворка

Команда	Назначение
atm run	Запуск одиночного эксперимента из YAML-конфига.
atm grid	Параллельный грид-эксперимент через ProcessPoolExecutor.
atm estimate	Предварительная оценка стоимости запуска по таблице цен и историческим данным.
atm status	Агрегированный статус эксперимента — число выполненных, неудачных и активных ячеек, суммарная стоимость.
atm resume	Возобновление прерванного запуска из последнего чекпойнта LangGraph.
atm replay	Детерминированное воспроизведение запуска через FakeLLM в режиме воспроизведения по записи.
atm reconcile	Поиск и пометка запусков-зомби (запись о статусе «выполняется», но процесс уже завершен).

Воспроизводимость обеспечивается набором инвариантов, фиксируемых в момент старта каждого запуска. Функция `seed_all` иницирует все известные источники случайности (Python, NumPy, провайдеры языковых моделей, генератор идентификаторов). Значение `git_sha` извлекается вызовом `git rev-parse HEAD` и записывается в таблицу экспериментов. Снимок версий используемых моделей формируется в первом успешном ответе от каждого провайдера и сохраняется в поле `model_version_snapshot` запуска. Дайджест образа Docker-песочницы фиксируется при ее инициализации. Совокупность этих полей делает возможным сравнение результатов между запусками и анализ артефактов депрекации моделей со стороны провайдеров.

3.7 Выводы по главе

Программная инфраструктура работы реализована как фреймворк из тринадцати функциональных слоев на языке Python с акцентом на расширяемость через протоколы, воспроизводимость через фиксацию инвариантов состояния и наблюдаемость через единый перехват событий LangGraph. В рамках этой инфраструктуры представлены шесть топологий, два режима маршрутизации фаз, три режима маршрутизации топологий и два шлюза взаимо-

действия с человеком — то есть все компоненты, необходимые для выполнения экспериментов главы 4. Подробное разграничение между заимствованными и авторскими компонентами приведено в таблице 3.2 и сформулировано в виде отдельного раздела авторского вклада 5.5.

Объемные характеристики реализации. Кодовая база основного пакета `atm` содержит 21 327 строк кода на языке Python, распределенных по 112 программным модулям внутри тринадцати функциональных слоев (то есть в среднем порядка девяти модулей на слой). Конфигурация экспериментов задается через 43 файла формата YAML, из которых 24 описывают непосредственные параметры запусков серии E1–E4. Приведенные числовые характеристики получены прямым измерением исходной кодовой базы на момент сдачи работы; служебные файлы — модульные и интеграционные проверки, скрипты сборки, документация проекта — в подсчет не включены.

4 Экспериментальное исследование

Настоящая глава содержит результаты эмпирической проверки исследовательских вопросов (от англ. Research Questions, RQ) RQ1–RQ4, поставленных в разделе 2.7. Глава организована как последовательное изложение четырех основных экспериментов воронки E1–E4, двух подтверждающих прогонов на кросс-семейном рабочем агенте и сводного статистического анализа на основе непараметрического бутстрапа.

4.1 Условия проведения и общие характеристики

Все эксперименты выполнены в единой программной инфраструктуре, описанной в главе 3, в период с 16 по 17 мая 2026 года. В роли основной языковой модели (*worker*) для E1–E4 использовалась `cerebras:gpt-oss-120b` с параметром `reasoning_effort=low` для рабочих агентов и `high` для агента-критика. В роли судьи оценки качества выступала `openai:gpt-4.1-mini` с самосогласованностью из трех независимых вызовов и нулевой температурой выборки. Шлюз `HumanGateway` (компонент человека в контуре управления, от англ. Human-in-the-Loop, HITL) работал в режиме симулятора через ту же `openai:gpt-4.1-mini` с системной подсказкой, зависящей от текущей роли. Подтверждающие прогоны (`confirmation_e3`, `confirmation_e4`) выполнены с заменой рабочего агента на `cerebras:qwen-3-235b-a22b-instruct-2507` (семейство Alibaba Qwen) при сохранении остального стека неизменным.

Сводные объемные характеристики экспериментальной серии приведены в таблице 4.1. Совокупный объем составил 5175 прогонов и суммарную стоимость около \$87 — существенно ниже планового бюджета в \$200 благодаря низкой удельной стоимости вызовов на инфраструктуре Cerebras (от англ. Wafer-Scale Engine, WSE). Во всех финальных грид-прогонах доля отказов нулевая.

Таблица 4.1. Объемные характеристики экспериментальной серии

Этап	Прогоны	Стоимость, \$	Время	Назначение
E1	900	12,62	3,7 ч	Базовая линия статических топологий, RQ1.
E2	2295	42,37	13,9 ч	Влияние роли человека по матрице (топология × роль), RQ3.
E3	540	10,12	12,0 ч	Адаптивная мета-топология в трех режимах маршрутизации, RQ2.
E4	540	6,38	2,1 ч	Адаптивная роль на уже-адаптивной топологии, RQ4.
conf_e3	360	6,50	1,0 ч	Кросс-семейная валидация E3 на семействе Qwen.
conf_e4	540	9,40	1,3 ч	Кросс-семейная валидация E4 на семействе Qwen.
Итого	5175	87,39	≈34 ч CPU	—

4.2 Эксперимент E1 — базовая линия статических топологий

Эксперимент E1 устанавливает верхнюю границу качества для пяти статических топологий на четырех корпусах задач — программирование (HumanEval), математические рассуждения (GSM8K), творческая генерация (CommonGen) и анализ данных (InfiAgent-DABench), далее DABench, — отвечая на исследовательский вопрос RQ1. Сетка прогона составила 5 топологий × 4 корпуса × 15 задач × 3 зерна = 900 прогонов; роль человека отключена (`human.role=none`). Результаты усреднения по ячейкам (топология × корпус) приведены в таблице 4.2.

Таблица 4.2. Среднее качество q статических топологий по корпусам (жирным выделен победитель в столбце), E1

Топология	HumanEval	GSM8K	CommonGen	DABench	Среднее
Chain	0,933	0,911	0,526	0,333	0,676
Star	0,911	1,000	0,427	0,282	0,655
Debate	0,644	0,933	0,584	0,348	0,627
Hierarchical	0,889	0,978	0,486	0,267	0,655
Mesh	0,667	0,867	0,397	0,319	0,562
<i>Среднее по победителям</i>	<i>0,933</i>	<i>1,000</i>	<i>0,584</i>	<i>0,348</i>	0,716

Содержательный итог. Победители на четырех корпусах различают-

ся — chain на HumanEval, star на GSM8K и debate на CommonGen и DABench. Единой топологии, доминирующей на всех типах задач, нет. Лучшая глобальная статика — chain со средним $q = 0,676$ — проигрывает идеальному выбору *под задачу* (среднее по победителям, $q = 0,716$) около четырех процентных пунктов. Этот разрыв задает *class-level upper bound* для последующего эксперимента E3.

Обнаружено два дополнительных явления. Во-первых, топология Mesh деградирует на программировании и творческой генерации до $q \leq 0,40$ из-за вырождения голосования по совпадению строк. Во-вторых, на корпусе DABench все статические топологии демонстрируют $q \leq 0,35$ при доле полностью неудачных прогонов 62–71 % — проявление, которое в разделе 4.6 будет переинтерпретировано как специфика рабочего агента, а не корпуса.

Ответ на RQ1. Универсального статического победителя не существует. Оптимальный выбор топологии — функция класса задач, что мотивирует ввод адаптивной мета-топологии в E3.

4.3 Эксперимент E2 — роль человека (RQ3)

Эксперимент E2 измеряет влияние роли человека в графе агентов поверх статических топологий. Сетка построена как декартово произведение 5 топологий $\times$ 5 ролей $\times$ 4 корпуса $\times$ 15 задач $\times$ 3 зерна с двумя фильтрами — по per-task top-3 топологий из E1 и по трем структурно бессмысленным парам (Debate $\times$ Coordinator, Chain $\times$ Peer, Hierarchical $\times$ Peer). Итоговый объем составил 2295 прогонов. Полная матрица победителей роли по корпусам приведена в таблице 4.3.

Таблица 4.3. Победитель среди пяти ролей человека по каждому корпусу с маргинализацией по топологиям, E2

Корпус	Победившая роль	q победителя	n ячеек	Δq vs E1 best static
HumanEval	Coordinator	0,948	135	+0,015
GSM8K	Monitor	0,963	135	−0,015
CommonGen	Peer	0,643	45	+0,059
DABench	Peer	0,563	90	+0,215

Совокупный лифт от подключения человека (*HITL lift*), рассчитанный на пересечении ячеек E1 и E2, составил +0,033 по среднему качеству (+4,8 % относительного прироста). Распределение лифта по ячейкам неоднородно. Положительный лифт концентрируется на трех конфигурациях — Hierarchical $\times$ CommonGen (+0,107), Chain $\times$ DABench (+0,087) и Mesh $\times$ DABench (+0,052); отрицательный лифт наблюдается только на Star $\times$ GSM8K (−0,062), где топология уже близка к потолку качества.

Ответ на RQ3. Подключение человека дает скромное улучшение среднего качества при существенно различном эффекте по ячейкам. Никакая единая роль не выигрывает на всех корпусах — ключевой аргумент против фиксированного выбора роли и в пользу адаптивной маршрутизации, проверяемой в E4.

4.4 Эксперимент E3 — адаптивная мета-топология (RQ2)

Эксперимент E3 непосредственно отвечает на центральный исследовательский вопрос RQ2. На единой адаптивной мета-топологии сравниваются три режима маршрутизатора — rule (детерминированное правилковое решение), llm (вызов языковой модели с парсингом по Pydantic) и oracle (чтение таблицы best per task из файла, построенного по результатам E1). Каждый режим прогнан в сетке 4 корпуса $\times$ 15 задач $\times$ 3 зерна = 180 прогонов; фиксирована роль человека Reviewer. Сводные показатели по режимам приведены в таблице 4.4.

Численная картина допускает двойственную интерпретацию. По абсолютному качеству все три режима маршрутизатора уступают лучшей статической конфигурации под корпус — оракул отстает на 1,9 процентных пункта, правилковой — на 7,2, маршрутизатор на основе языковой модели — на 8,5. Причина — организационная накладная стоимость адаптивной обвязки (фазы $\times$ подграфы $\times$ маршрутизатор), которая не окупается на хорошо-решаемых корпусах (GSM8K, HumanEval). Единственный корпус с положительным абсолютным эффектом адаптации — CommonGen, где оракул дает +0,07 относительно лучшей статики.

Таблица 4.4. Сводные показатели трех режимов маршрутизатора топологий, E3

Режим	Среднее q	Стоимость, \$	Время, с	Итераций	Комментарий
oracle	0,697	0,0227	1077	6,27	Верхняя граница, знание корпуса.
rule	0,645	0,0236	928	6,04	Правилковый, без вызовов языковой модели.
llm	0,631	0,0099	881	6,10	Вызов языковой модели на каждый переход.
<i>Best static (E1)</i>	<i>0,716</i>	<i>0,014</i>	<i>448</i>	<i>4,35</i>	<i>Идеальный выбор статике под корпус.</i>

Сравнение трех режимов маршрутизатора между собой — *внутри* адаптивного класса — выявляет дополнительный неосновной эффект. Маршрутизатор на основе языковой модели проигрывает правилковому всего 1,3 процентных пункта по качеству, но стоит в 2,38 раза дешевле. Соотношение стоимостей удерживается на всех четырех корпусах в диапазоне 2,12–3,00, что указывает на *структурный* эффект — маршрутизатор на основе языковой модели систематически выбирает более дешевые подтопологии (например, Hierarchical с max_rounds=2) против более прожорливых решений правилковой таблицы. Этот сопутствующий эффект не отвечает на RQ2, но представляет самостоятельный интерес и обсуждается отдельно в разделе 4.6.

Ответ на RQ2. Адаптивная мета-топология не дает значимого прироста качества и не дает экономии стоимости по сравнению с лучшей статической конфигурацией под корпус. В качестве референса для сравнения используется не одна фиксированная статика, а оракул-выбор наилучшей статической топологии для каждого корпуса (среднее качество 0,716). Оракул адаптивной обвязки достигает 0,697 и отстает от этого референса на 1,9 процентных пункта; правилковой и языковой маршрутизаторы — на 7,2 и 8,5 процентных пункта. Гипотеза RQ2 эмпирически не подтверждается. Обнаруженная рядом сопутствующая находка — двукратная экономия стоимости маршрутизатора на основе языковой модели *внутри* адаптивного класса — не отвечает на RQ2

в его исходной постановке (сравнение адаптивной и статической конфигураций), и при кросс-семейной валидации на семействе Qwen, где правилевой маршрутизатор Парето- доминирует над маршрутизатором на основе языковой модели, эта находка также не воспроизводится (см. 4.6).

4.5 Эксперимент E4 — адаптивная роль (RQ4)

Эксперимент E4 размещает второй слой адаптивности — маршрутизатор ролей — поверх уже-адаптивной топологии. На основе результатов E3 зафиксирован выбор `topology_router=llm` как Парето-оптимальной точки адаптивного класса. Считаемая переменная — `role_router` $\in \{\text{fixed}, \text{rule}, \text{llm}\}$; сетка $3 \times 4 \times 15 \times 3 = 540$ прогонов. Сводка по режимам — в таблице 4.5.

Таблица 4.5. Сводные показатели трех режимов маршрутизатора ролей поверх адаптивной мета-топологии, E4

Режим	Среднее q	Δq vs fixed	Стоимость, \$	Время, с
fixed	0,653	—	0,01186	381
rule	0,680	+0,027	0,01193	356
llm	0,643	−0,010	0,01168	285

Маршрутизатор на основе правил формально увеличивает среднее качество на 2,7 процентных пункта над фиксированной ролью `reviewer`. Направление совпадает на четырех корпусах из четырех при оговорке о малом размере выборки и одном семействе весов рабочего агента; драйвер прироста — HumanEval (+0,067). Стоимость трех режимов укладывается в коридор шириной 0,85 % — дополнительные стоимостные издержки адаптивной маршрутизации роли незначимы.

Качественный анализ распределения ролей показал, что правилевой маршрутизатор фактически коллапсирует в единственную роль — координатор активизируется в 98,3 % переходов. То есть адаптации роли по фазам в текущей реализации не происходит — фактически реализуется почти-статическая политика с единственной ролью. E4 переоткрывает результат E2 «Coordinator выигрывает на HumanEval» в новой обертке. Подробная интерпретация этого механизма дана в разделе 5.1.

Ответ на RQ4. Гипотеза о превосходстве совместной адаптации топологии и роли над статическими комбинациями эмпирически не подтверждается. На исходной семье моделей правиловая маршрутизация роли дает направленный прирост качества $+0,027$ при нулевой дополнительной стоимости, однако бутстрап-доверительный интервал $[-0,055, +0,110]$ покрывает нуль, и строгий вывод о значимости прироста не делается. Сопутствующая находка — направление эффекта совпадает на всех четырех корпусах из четырех — при кросс-семейной валидации (4.6) также не подтверждается — на семействе Qwen знак эффекта становится отрицательным.

4.6 Подтверждающие прогоны — кросс-семейная валидация

Два подтверждающих прогона — `confirmation_e3` и `confirmation_e4` — повторяют экспериментальную сетку E3 и E4 с единственной измененной переменной — семейством рабочей языковой модели. Вместо `gpt-oss-120b` использована `qwen-3-235b-a22b-instruct-2507` (семейство Alibaba Qwen). Все остальные компоненты (судья, шлюз HITL, инфраструктура, адаптивная обвязка, защитные правила, конфигурация порогов) сохранены неизменными. Цель — проверить переносимость основных выводов E3 и E4 между семействами весов.

Результаты `confirmation_e3` (`topology_router` $\in$ $\{\text{rule, llm}\}$) приведены в таблице 4.6. Соотношение стоимостей правилового маршрутизатора к маршрутизатору на основе языковой модели схлопывается с 2,38 на исходном семействе до 1,11 на семействе Qwen, а разрыв качества увеличивается с +1,3 процентных пункта до +30,6 в пользу правилового режима.

Таблица 4.6. Сравнение двух режимов маршрутизатора топологий между семействами весов, `confirmation_e3`

Семейство	q_{rule}	q_{llm}	Δq	$c_{\text{rule}}/c_{\text{llm}}$	Парето-победитель
<code>gpt-oss-120b (E3)</code>	0,645	0,631	+0,013	2,38	llm (по стоимости)
<code>qwen-3-235b (conf_e3)</code>	0,866	0,560	+0,307	1,11	rule (по обеим осям)

Аналогичная картина для `confirmation_e4` (см. таблицу 4.7). На исходном семействе моделей правилowej маршрутизатор роли превосходил фиксированную конфигурацию на $+0,027$; на семействе Qwen он, напротив, уступает ей на $-0,081$ — знак эффекта изменился.

Таблица 4.7. Сравнение трех режимов маршрутизатора ролей между семействами весов, `confirmation_e4`

Режим	q на gpt-oss	q на qwen	Δ vs fixed gpt-oss	Δ vs fixed qwen
fixed	0,653	0,582	—	—
rule	0,680	0,501	$+0,027$	$-0,081$
llm	0,643	0,528	$-0,010$	$-0,054$

Качественный анализ выявил дополнительный эффект — сдвиг распределения ролей при неизменной таблице `DEFAULT_ROLE_TABLE`. На исходной семье правилowej маршрутизатор выбирал *координатор* в 98 % переходов; на семье Qwen тот же маршрутизатор с той же таблицей выбирает *равноправного участника* в 94 % переходов. Иначе говоря, политика, выглядящая независимой от модели, фактически зависит от нее через входы — фазовые классификаторы получают разные сигналы от моделей разных семейств и относят те же ситуации к разным фазам.

Дополнительная находка — DABench. На исходном семействе корпус DABench демонстрировал среднее качество $0,15\text{--}0,24$ во всех экспериментах E1–E4; на семье Qwen те же конфигурации дают качество $0,84\text{--}0,93$. Это согласуется с гипотезой о том, что DABench измеряет компетенции, специфичные для семейства весов рабочего агента, а не является «сломанным корпусом»; численный контраст между семействами — порядка 0,7 процентных пункта — исключает корпус как источник сквозного смещения.

Ограничение интерпретации. На исходном семействе работники запускались с параметром `reasoning_effort=low`; программный интерфейс Qwen этот параметр не принимает. Часть наблюдаемых кросс-семейных различий смешана с различием в глубине рассуждений работника и не отделяется от эффекта семейства весов без дополнительного контроля; это явное ограничение, требующее повторной серии при появлении соответствующей опции в ин-

терфейсе Qwen.

4.7 Бутстрап-доверительные интервалы

Для оценки устойчивости заявленных эффектов проведен непараметрический бутстрап с десятью тысячами итераций (метод процентилей, $\text{seed}=42$). Сводные интервалы для ключевых разностей и отношений — в таблице 4.8.

Таблица 4.8. Бутстрап-доверительные интервалы 95 % для ключевых разностей и отношений

Сравнение	Точечная оценка	Интервал 95 %	Содержит границу?
E3 rule—llm, q (gpt-oss)	+0,013	$[-0,072, +0,097]$	да (нуль)
E3 oracle—rule, q (gpt-oss)	+0,053	$[-0,033, +0,137]$	да (нуль)
E3 $c_{\text{rule}}/c_{\text{llm}}$	2,38	$[1,94, 2,94]$	нет (единицу)
E4 rule—fixed, q (gpt-oss)	+0,027	$[-0,055, +0,110]$	да (нуль)
E4 rule—fixed (HumanEval, gpt-oss)	+0,067	$[-0,022, +0,156]$	да (нуль)
conf_e3 rule—llm, q (qwen)	+0,307	$[+0,233, +0,378]$	нет (нуль)
conf_e3 $c_{\text{rule}}/c_{\text{llm}}$ (qwen)	1,11	$[0,98, 1,25]$	да (единицу)
conf_e4 rule—fixed, q (qwen)	−0,081	$[-0,173, +0,010]$	пограничное

Интервалы позволяют отделить устойчивые эффекты от направленных точечных оценок. Устойчивых эффектов в работе три. Во-первых, двукратная экономия стоимости маршрутизатора на основе языковой модели на исходной семье — интервал отношения стоимостей $[1,94, 2,94]$ никогда не пересекает единицу. Во-вторых, изменение знака эффекта качества правил маршрутизатора при смене семьи весов — интервал разности качеств на семье Qwen $[+0,233, +0,378]$ устойчиво исключает нуль. В-третьих, схлопывание соотношения стоимостей при смене семьи — интервал $[0,98, 1,25]$ покрывает единицу, что указывает на структурный характер схлопывания. Остальные точечные оценки направленные, но не устойчивы — бутстрап-интервалы включают нуль, и эффекты в строгом смысле статистически неотличимы от нулевого.

4.8 Абляционное исследование

Поскольку структура воронки от E1 до E4 уже устроена как последовательное *добавление* компонентов — сначала статические топологии, затем человек, затем адаптивная топология, затем адаптивная роль — сама воронка играет роль абляционного исследования. Сводка эффектов компонентов приведена в таблице 4.9.

Таблица 4.9. Абляционная сводка — эффект добавления компонента относительно базовой линии

Компонент	Δq	Δc	Источник эффекта
Best static per task (E1)	—	—	Базовая линия.
+ HITL фиксированной роли (E2)	+0,033	+0,004	Подключение симулятора человека.
+ Адаптивная мета-топология, oracle (E3)	−0,019	+0,008	Накладные расходы маршрутизации без явного выигрыша.
+ Адаптивная роль, rule (E4 vs E3)	+0,049	≈ 0	Скрытое смещение к роли <i>coordinator</i> .
+ Смена семейства модели (conf_e4 vs E4)	−0,108	+0,005	Падение знаком и magnitude — переносимость не подтверждена.

Содержательно абляция показывает, что только два компонента — подключение человека (E2 vs E1) и фиксация роли *coordinator* через правиловую таблицу (E4 vs E3) — дают положительное приращение качества; адаптивная топология сама по себе ухудшает качество относительно идеально-настроенной статистики; смена семейства весов меняет знак эффекта роли. Кросс-семейная неустойчивость ставит под вопрос интерпретацию положительных приращений как универсальных, а не специфичных для исходной семьи моделей.

4.9 Выводы по главе

Результаты экспериментальной серии сводятся к четырем содержательным итогам. Первый — класс задач определяет оптимальную статическую топологию (RQ1 подтверждается). Второй — адаптивная мета-топология не превосходит лучшую статическую конфигурацию под корпус ни по каче-

ству, ни по стоимости (RQ2 не подтверждается); рядом обнаружен неосновной структурный эффект — двукратная экономия стоимости маршрутизатора на основе языковой модели внутри адаптивного класса — который при кросс-семейной валидации также не подтверждается. Третий — подключение человека дает скромный направленный прирост качества при существенно разном эффекте по ячейкам (RQ3 подтверждается). Четвертый — совместная адаптация топологии и роли не превосходит фиксированные комбинации (RQ4 не подтверждается); направление эффекта совпадает на четырех корпусах из четырех в исходной семье моделей, но при кросс-семейной валидации знак меняется и эффект исчезает.

Единственный устойчивый кросс-семейный вывод — зависимость оптимальной политики маршрутизации от семейства весов рабочего агента. Его содержательная интерпретация и обсуждение угроз валидности вынесены в главу 5.

5 Анализ результатов и обсуждение

Настоящая глава содержит обобщающий анализ экспериментальных результатов главы 4, последовательные ответы на четыре исследовательских вопроса, формулировку центрального вывода работы и обсуждение угроз валидности.

5.1 Ответы на исследовательские вопросы

RQ1 — статические топологии и класс задач. Гипотеза о зависимости оптимальной статической топологии от класса задач эмпирически подтверждается. Победители на четырех корпусах — три различные топологии (Chain — на HumanEval с $q = 0,933$; Star — на GSM8K с $q = 1,000$; Debate — на CommonGen и DABench с $q = 0,584$ и $q = 0,348$). Лучшая глобальная статическая конфигурация (Chain со средним $q = 0,676$) уступает оракулу-выбору статике под корпус ($q = 0,716$) на четыре процентных пункта — эта разность задает референс для последующего сравнения адаптивных конфигураций.

RQ2 — адаптивная мета-топология. Гипотеза о превосходстве адаптивной мета-топологии над лучшей статической конфигурацией под корпус эмпирически не подтверждается. В качестве референса используется оракул-выбор лучшей статической топологии для каждого корпуса со средним качеством 0,716 при стоимости 0,014 (одна и та же топология, своя для каждого корпуса, без переключений во время выполнения задачи). Оракул адаптивной обвязки достигает $q = 0,697$, отставая от этого референса на 1,9 процентных пункта; правилковой и языковой маршрутизаторы отстают от оракула адаптивной обвязки на 5,3 и 6,6 процентных пункта. По стоимости адаптивная обвязка также не превосходит референс — самый дешевый адаптивный режим (маршрутизатор на основе языковой модели) дает 0,631 при 0,010, теряя 8,5 процентных пункта качества ради 30 % экономии стоимости. Причина — организационные накладные расходы адаптивной обвязки (переходы фаз, переключения подграфов, обращения к маршрутизатору), которые

не окупаются на корпусах с высоким абсолютным качеством работы статистики (HumanEval, GSM8K). Положительный эффект адаптации концентрируется на CommonGen, где творческий характер задач оправдывает разнообразие подграфов.

Рядом с отрицательным ответом на RQ2 обнаружена самостоятельная сопутствующая находка — внутри адаптивного класса маршрутизатор на основе языковой модели уступает правилowому всего 1,3 процентных пункта качества, будучи в 2,38 раза дешевле; точечная оценка отношения стоимостей устойчиво находится в интервале $[1,94, 2,94]$ и никогда не приближается к единице. Соотношение стоимостей сохраняется на всех четырех корпусах в диапазоне 2,12–3,00, что указывает на *структурный* характер эффекта — маршрутизатор на основе языковой модели систематически выбирает дешевые подтопологии при сохранении уровня качества. Эта сопутствующая находка не отвечает на RQ2 в его исходной постановке (сравнение адаптивной и статической конфигураций), и при кросс-семейной валидации она также не подтверждается — на семье Qwen соотношение стоимостей коллапсирует до 1,11 при одновременном падении качества маршрутизатора на основе языковой модели на 30,6 процентных пункта.

RQ3 — роль человека. Подключение человека к контуру управления дает положительный направленный лифт в среднем +0,033 (относительный прирост +4,8 %) при существенной неоднородности по ячейкам. Победители роли различаются по корпусам — Coordinator — на HumanEval, Monitor — на GSM8K, Peer — на CommonGen и DABench. В разрезе топологий лучшая роль также не единая — например, на Star лучше всего работают Monitor и Reviewer (оба с $q = 0,944$), на Hierarchical — Coordinator ($q = 0,842$), на Mesh — Peer ($q = 0,600$). Положительный лифт выше пяти процентных пунктов наблюдается на трех конфигурациях — Hierarchical $\times$ CommonGen, Chain $\times$ DABench и Mesh $\times$ DABench. Эти результаты дают эмпирический материал для последующей проверки адаптивной маршрутизации роли в E4.

RQ4 — адаптивная роль. Гипотеза о превосходстве совместной адаптации топологии и роли над статическими комбинациями эмпирически не

подтверждается. В исходной семье моделей правилочная маршрутизация роли формально увеличивает среднее качество на $+0,027$ относительно фиксированной конфигурации, однако бутстрап-доверительный интервал $[-0,055, +0,110]$ покрывает нуль, и утверждение о значимости прироста по строгому критерию не делается. Сопутствующая находка — направление эффекта совпадает единообразно на четырех корпусах из четырех в исходной семье моделей — при кросс-семейной валидации не подтверждается — на семействе Qwen знак эффекта меняется на противоположный с $-0,081$. На двух семьях моделей правилочной маршрутизатор с идентичной таблицей `DEFAULT_ROLE_TABLE` коллапсирует в разные роли — координатор в 98 % переходов на семье gpt-oss и равноправный участник в 94 % на семье Qwen. Это указывает на зависимость «правилочной» политики от базовой модели через входы — фазовые классификаторы относят те же ситуации к разным фазам в зависимости от поведения рабочего агента, что делает заявление об универсальности правил маршрутизации некорректным.

5.2 Главный нарратив исследования

Единственным эффектом, прошедшим бутстрап-валидацию на обеих рассмотренных в работе семьях весов рабочего агента, является *зависимость оптимальной политики маршрутизации от семейства весов рабочего агента*. Этот вывод имеет три проявления, последовательно подтвержденных бутстрап-анализом.

Первое — разворот Парето-победителя. На семействе gpt-oss маршрутизатор на основе языковой модели находится на границе Парето (двукратная экономия стоимости при квазипаритете качества); на семействе Qwen на Парето-границе находится правилочной маршрутизатор (он одновременно выше по качеству и дешевле на единицу качества). Доверительный интервал разности качеств правилочного и языкового маршрутизаторов на семействе Qwen $[+0,233, +0,378]$ устойчиво исключает нуль.

Второе — схлопывание структуры стоимостей. На исходной семье отношение стоимостей правилочного и языкового маршрутизаторов устойчиво 2,38

и не пересекает единицу. На семействе Qwen то же отношение коллапсирует до 1,11 с доверительным интервалом $[0,98, 1,25]$, покрывающим единицу. Изменение порядка стоимостей по семейству — структурное, а не случайное.

Третье — сдвиг распределения ролей. Та же правилочная таблица, тот же кодовый путь маршрутизатора фаз, та же языковая модель шлюза человека — на двух семействах рабочего агента дают качественно разные роли в 94 % переходов. Это поведенческое расхождение перекладывает источник «разумности» адаптивных политик с таблиц правил на поведение базовой модели, получающей входы от правил.

Содержательно центральный вывод формулируется так — адаптивные политики маршрутизации в мульти-агентных системах больших языковых моделей, выглядящие независимыми от базовой модели через таблицы правил или промпты, фактически зависят от нее через свои входы. Универсальные заявления о преимуществах адаптивной маршрутизации, полученные на одном семействе весов, без кросс-семейной валидации эпистемологически слабы.

5.3 Сопоставление с гипотезами

Сопоставление полученных результатов с гипотезами, сформулированными в разделе 2, приведено в таблице 5.1.

5.4 Угрозы валидности и ограничения

Работа имеет ряд ограничений, которые ограничивают область обобщения выводов.

Во-первых, размер выборки — $n = 180$ прогонов на ячейку эксперимента E3 и E4 — недостаточен для устойчивой проверки эффектов меньше пяти процентных пунктов. Все направленные приросты качества в работе укладываются именно в этот диапазон, и бутстрап-доверительные интервалы соответственно покрывают нуль. Устойчивыми оказываются только эффекты, превышающие ± 25 процентных пунктов (разворот качества при смене семейства).

Таблица 5.1. Сопоставление гипотез работы и эмпирических результатов

Гипотеза	Эмпирический статус
Н1: класс задач определяет лучшую статическую топологию	Подтверждено
Н2: адаптивная мета-топология превосходит лучшую статическую конфигурацию под корпус	Не подтверждено
Н3: подключение человека увеличивает среднее качество	Подтверждено

Во-вторых, кросс-семейная валидация выполнена только на двух семействах весов — gpt-oss и Qwen. Для усиления заявления о семейной зависимости политик маршрутизации требуется как минимум еще одно-два семейства (Claude, Gemini, LLaMA-4).

В-третьих, на подтверждающих прогонах был отключен параметр `reasoning_eff` поскольку интерфейс семейства Qwen его не поддерживает. Это вносит неустрашимое смещение между эффектом семейства весов и эффектом глубины рассуждений рабочего агента. Часть наблюдаемых кросс-семейных различий (особенно крупных падений качества на GSM8K и HumanEval при смене семейства) не отделяется чисто от смешанной причины без дополнительной серии с совпадающим параметром глубины рассуждений.

В-четвертых, человек в контуре управления моделируется языковой моделью с зависимой от роли системной подсказкой. Эмпирическая проверка соответствия симулятора и реального участника не выполнена — эта проверка вынесена в следующий этап исследования (E5) и не входит в настоящую работу. Все выводы об эффекте ролей относятся к симуляции и не переносятся напрямую на реальных участников без подтверждения.

В-пятых, корпус задач состоит из четырех открытых наборов, по одному на класс. Стратифицированная выборка — 15 задач из каждого корпуса — обеспечивает воспроизводимость, но ограничивает обобщение на непокрытые классы (диалог, перевод, обобщение длинных документов).

В-шестых, судья оценки качества — единственная модель из единственного семейства весов (OpenAI gpt-4.1-mini). Подтверждение независимости выводов от смещения судьи требует параллельной оценки на судье из другого семейства, что выходит за рамки настоящей работы.

В-седьмых, защитные правила переключения топологий в текущей реализации существенно сужают пространство фактических переключений — правилевой маршрутизатор роли коллапсирует в единственную роль в 94–98 % переходов. Это означает, что «подлинно фазовой» адаптивности в экспериментах E4 не происходит — эффект сводится к неявному выбору единственной лучшей роли через правилевую таблицу. Эта особенность зафиксирова-

на как известное ограничение реализации и требует ослабления защитных правил в последующих прогонах для проверки изначальной гипотезы о фазовой адаптации.

5.5 Выводы по главе

Из четырех исследовательских вопросов работы непосредственный положительный ответ получен на два — RQ1 (класс задач определяет оптимальную статическую топологию) и RQ3 (подключение человека дает направленный лифт качества). Гипотезы RQ2 и RQ4 эмпирически не подтверждаются — ни адаптивная мета-топология, ни совместная адаптация топологии и роли не превосходят лучшую статическую конфигурацию по ключевым показателям, на которые сформулированы вопросы. Рядом с отрицательными ответами обнаружены две сопутствующие находки — двукратная экономия стоимости маршрутизатора на основе языковой модели внутри адаптивного класса (RQ2-сторона) и направленный прирост качества от правильной маршрутизации роли в исходной семье моделей (RQ4-сторона). Обе сопутствующие находки не отвечают на исходно поставленные вопросы и при кросс-семейной валидации не подтверждаются.

Центральный вывод работы — зависимость оптимальной политики маршрутизации от семейства весов рабочего агента — является единственным эффектом, устойчиво подтвержденным бутстрап-доверительными интервалами в кросс-семейной постановке, и формулируется как методологическое утверждение — любые претензии на универсальность адаптивных политик в мульти-агентных системах больших языковых моделей требуют валидации не менее чем на двух семействах весов рабочего агента.

Авторский вклад

Авторский вклад настоящей работы сводится к четырем результатам.

Первый — программная инфраструктура исследования. В рамках работы с нуля разработан и опубликован под лицензией MIT мульти-агентный

фреймворк ATM (Adaptive Topologies for Multi-agent systems), объединяющий пять статических коммуникационных топологий, адаптивную мета-топологию второго уровня, конечный автомат фаз, три режима маршрутизатора фаз и три режима маршрутизатора топологий, защитные правила переключения, шлюз взаимодействия с человеком с пятью ролями и маршрутизатором ролей, оракул-маршрутизатор и подсистему наблюдаемости с двухкомпонентным хранилищем (реляционная база PostgreSQL и колоночный формат Apache Parquet). Кодовая база содержит 21 327 строк кода на языке Python, распределенных по 112 программным модулям внутри тринадцати функциональных слоев.

Второй — формальная модель и таксономия метрик. Введена формализация мульти-агентной системы как кортежа из шести компонентов с зависящей от фазы коммуникационной структурой (формулы 2.2 и 2.3). Введена таксономия из пяти адаптивно-специфичных метрик — число переключений топологии, доля заблокированных решений маршрутизатора, стоимостный вклад маршрутизатора, доля регрессий качества и разрыв до оракула (формула 2.4). Введена прокси-метрика когнитивной нагрузки для запусков с симулятором (формула 2.5). Совокупность метрик позволяет количественно характеризовать поведение динамически перестраиваемых мульти-агентных систем и сопоставлять их по единой шкале.

Третий — эмпирические результаты. Проведена воронка из четырех основных экспериментов и двух кросс-семейных подтверждающих прогонов общим объемом 5175 прогонов с нулевой долей отказов в финальных грид-сериях. Для каждого исследовательского вопроса получен явный количественный ответ — по RQ1 и RQ3 ответы положительные (универсального статического победителя нет; оптимальная роль человека зависит от корпуса и топологии), по RQ2 и RQ4 ответы отрицательные (ни адаптивная мета-топология, ни совместная адаптация топологии и роли не превосходят референс). Рядом с отрицательными ответами обнаружены две сопутствующие находки — двукратная экономия стоимости маршрутизатора на основе языковой модели внутри адаптивного класса и направленный прирост качества от правило-

вой маршрутизации роли в исходной семье моделей — которые не отвечают на исходно поставленные вопросы и при кросс-семейной валидации также не подтверждаются. Все головные эффекты сопровождаются непараметрическими бутстрап-доверительными интервалами.

Четвертый, ключевой — методологический вывод. Установлена зависимость оптимальной политики маршрутизации в адаптивных мульти-агентных системах от семейства весов рабочего агента и продемонстрировано, что «правилковые» политики, выглядящие независимыми от модели через таблицы, фактически зависят от нее через свои входы (классификаторы фаз). На основании этого наблюдения сформулировано методологическое требование — любые претензии на универсальность адаптивных политик в мульти-агентных системах больших языковых моделей требуют валидации не менее чем на двух семействах весов рабочего агента. Это требование меняет статус кросс-семейной валидации с необязательного дополнения на обязательный шаг при исследовании адаптивных мульти-агентных систем.

Заключение

В настоящей выпускной квалификационной работе проведена количественная оценка влияния выбора коммуникационной топологии, механизма ее адаптации и позиции человека в контуре управления на эффективность решения задач разных классов мульти-агентной системой на основе больших языковых моделей. Цель работы достигнута — получены явные количественные характеристики для пяти статических топологий, адаптивной мета-топологии в трех режимах маршрутизации, пяти ролей человека и их адаптивной комбинации; всего выполнено 5175 прогонов на четырех корпусах задач с суммарной стоимостью около \$87.

На исследовательский вопрос RQ1 получен положительный ответ — универсальной статической топологии-доминатора не существует; победители на четырех корпусах различаются (Chain — на HumanEval, Star — на GSM8K, Debate — на CommonGen и DABench). На RQ3 также получен положительный ответ — подключение человека к контуру управления дает направленный прирост среднего качества $+0,033$ относительно конфигурации без человека при существенной неоднородности эффекта по ячейкам. На RQ2 получен отрицательный ответ — адаптивная мета-топология не дает значимого прироста качества и не дает устойчивой экономии стоимости по сравнению с лучшей статической конфигурацией под корпус; рядом с этим отрицательным ответом обнаружена сопутствующая находка — двукратная экономия стоимости маршрутизатора на основе языковой модели внутри адаптивного класса с бутстрап-доверительным интервалом $[1,94, 2,94]$ — которая не отвечает на RQ2 в его исходной постановке и при кросс-семейной валидации также не подтверждается. На RQ4 ответ также отрицательный — направленный прирост качества от адаптивной роли $+0,027$ в исходной семье моделей имеет бутстрап-доверительный интервал $[-0,055, +0,110]$, покрывающий нуль, а при кросс-семейной валидации на семействе Qwen знак эффекта меняется на $-0,081$.

Ключевой результат работы — кросс-семейная неустойчивость адаптив-

ных политик маршрутизации в мульти-агентных системах больших языковых моделей. Установлено, что таблица правил маршрутизатора и архитектура промптов маршрутизатора, выглядящие независимыми от базовой модели, фактически зависят от нее через свои входы — фазовые классификаторы получают разные сигналы от работников разных семейств весов и относят те же ситуации к разным фазам. Это переводит вопрос об универсальности адаптивных политик из технического в методологический и формирует требование к кросс-семейной валидации как обязательному шагу исследования.

Работа имеет шесть основных ограничений. Размер выборки $n = 180$ прогонов на ячейку недостаточен для проверки эффектов меньше пяти процентных пунктов; кросс-семейная валидация выполнена только на двух семействах весов; параметр глубины рассуждений работника смешан с эффектом семейства весов. Дополнительно, человек в контуре управления моделируется языковой моделью, что требует отдельной валидации на реальных участниках. Корпус задач состоит из четырех открытых наборов и не покрывает ряд важных классов (диалог, перевод, обобщение длинных текстов). Судья оценки качества — единственная модель из единственного семейства весов, что также требует параллельной оценки судьей из другого семейства для исключения смещения.

Направления дальнейших исследований

Прямое продолжение работы образует шесть взаимодополняющих направлений. Первое — расширение кросс-семейной валидации на три и более семейства весов рабочего агента (Anthropic Claude, Google Gemini, Meta LLaMA-4) с приведением параметра глубины рассуждений к единой шкале, что устранит смешение эффектов семейства и режима работы агента, ставшее основным ограничением подтверждающих прогонов настоящей работы. Это позволит проверить, является ли наблюдаемая кросс-семейная неустойчивость политик маршрутизации универсальным свойством адаптивных систем или специфичной для конкретной пары семейств. Второе — валидация симулятора человека на реальных участниках по схеме планируемого экспе-

римента E5 с использованием шкалы NASA-TLX (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA), латино-квадратного дизайна на восьми-двенадцати участниках и отобранного подмножества из двенадцати задач, что позволит численно оценить корреляцию между симулированной и фактической оценкой когнитивной нагрузки. Третье — разработка семейно-осознанного маршрутизатора, в котором политика выбора топологии и роли явно учитывает семейство весов рабочего агента, например, через предварительное дообучение маршрутизатора на трассах целевой модели или через переключение между семейно-специфичными политиками в зависимости от активного работника. Четвертое — расширение корпуса задач на непокрытые классы (диалог, перевод, обобщение длинных документов, мультимодальные задачи) и проверка обобщения результатов воронки на новые классы. Пятое — ослабление защитных правил переключения и проведение повторной серии E4 с целью выяснить, является ли наблюдаемое коллапсирование маршрутизатора роли в единственную роль артефактом текущих настроек или фундаментальным свойством адаптивной маршрутизации в исследуемом стеке. Шестое — параллельная оценка качества решений двумя судьями из разных семейств весов, что позволит численно отделить эффект адаптации от возможного смещения единственного судьи.

Библиографический список

- [1] A survey on large language model based autonomous agents / L. Wang, C. Ma, X. Feng [и др.] // *Frontiers of Computer Science*. – 2024. – Vol. 18, no. 6. – P. 186345.
- [2] AgentBench: Evaluating LLMs as agents [Электронный ресурс] / X. Liu, H. Yu, H. Zhang [и др.] // *arXiv preprint 2308.03688*. – 2023. – URL: <https://arxiv.org/abs/2308.03688> (дата обращения 17.05.2026).
- [3] AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors [Электронный ресурс] / W. Chen, Y. Su, J. Zuo [и др.] // *arXiv preprint 2308.10848*. – 2023. – URL: <https://arxiv.org/abs/2308.10848> (дата обращения 17.05.2026).
- [4] AutoGen: Enabling next-gen LLM applications via multi-agent conversation [Электронный ресурс] / Q. Wu, G. Bansal, J. Zhang [и др.] // *arXiv preprint 2308.08155*. – 2023. – URL: <https://arxiv.org/abs/2308.08155> (дата обращения 17.05.2026).
- [5] Benjamini Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing / Y. Benjamini, Y. Hochberg // *Journal of the Royal Statistical Society. Series B*. – 1995. – Vol. 57, no. 1. – P. 289–300.
- [6] CAMEL: Communicative agents for «mind» exploration of large language model society / G. Li, H. Hammoud, H. Itani, D. Khizbullin, B. Ghanem // *Advances in Neural Information Processing Systems*. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 51991–52008.
- [7] Chen L. FrugalGPT: How to use large language models while reducing cost and improving performance [Электронный ресурс] / L. Chen, M. Zaharia, J. Zou // *Transactions on Machine Learning Research*. – 2024. – URL: <https://openreview.net/forum?id=cSimKw5p6R> (дата обращения 17.05.2026).
- [8] ChatDev: Communicative agents for software development / C. Qian, W. Liu, H. Liu [и др.] // *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. – Stroudsburg, PA: ACL, 2024. – P. 15174–15186.
- [9] Chase H. LangChain: Building applications with LLMs through composability [Электронный ресурс] / H. Chase. – 2023. – URL: <https://github.com/langchain-ai/langchain> (дата обращения 17.05.2026).
- [10] Christiano P. F. Deep reinforcement learning from human preferences / P. F. Christiano, J. Leike, T. Brown [и др.] // *Advances in Neural Information Processing Systems*. – Red Hook, NY: Curran Associates, 2017. – Vol. 30. – P. 4299–4307.
- [11] CommonGen: A constrained text generation challenge for generative commonsense reasoning / B. Y. Lin, W. Zhou, M. Shen [и др.] // *Findings of the Association for Computational Linguistics (EMNLP 2020)*. – Stroudsburg, PA: ACL, 2020. – P. 1823–1840.
- [12] Cohen J. Statistical power analysis for the behavioral sciences / J. Cohen. – 2nd ed. – Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. – 567 p.
- [13] Colvin S. Pydantic: Data validation using Python type hints [Электронный ресурс] / S. Colvin [и др.]. – 2024. – URL: <https://docs.pydantic.dev/> (дата обращения 17.05.2026).
- [14] Efron B. Bootstrap methods: another look at the jackknife / B. Efron // *The Annals of Statistics*. – 1979. – Vol. 7, no. 1. – P. 1–26.
- [15] Evaluating large language models trained on code [Электронный ресурс] / M. Chen, J. Tworek, H. Jun [и др.] // *arXiv preprint 2107.03374*. – 2021. – URL: <https://arxiv.org/abs/2107.03374> (дата обращения 17.05.2026).
- [16] G-Designer: Architecting multi-agent communication topologies via graph neural networks [Электронный ресурс] / Y.

- Zhang, K. Xu, Y. Li, Z. Liu, D. Shen // arXiv preprint 2410.11782. – 2024. – URL: <https://arxiv.org/abs/2410.11782> (дата обращения 17.05.2026).
- [17] GPTSwarm: Language agents as optimizable graphs / M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, J. Schmidhuber // Proceedings of the International Conference on Machine Learning (ICML 2024). – Cambridge, MA: PMLR, 2024. – P. 62556–62583.
- [18] Hart S. G. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research / S. G. Hart, L. E. Staveland // Human Mental Workload / ed. by P. A. Hancock, N. Meshkati. – Amsterdam: North-Holland, 1988. – P. 139–183.
- [19] HellaSwag: Can a machine really finish your sentence? / R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, Y. Choi // Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019). – Stroudsburg, PA: ACL, 2019. – P. 4791–4800.
- [20] Horvitz E. Principles of Mixed-Initiative User Interfaces / E. Horvitz // Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99). – New York: ACM, 1999. – P. 159–166.
- [21] Improving factuality and reasoning in language models through multiagent debate [Электронный ресурс] / Y. Du, S. Li, A. Torralba, J. Tenenbaum, I. Mordatch // arXiv preprint 2305.14325. – 2023. – URL: <https://arxiv.org/abs/2305.14325> (дата обращения 17.05.2026).
- [22] InfiAgent-DABench: Evaluating agents on data analysis tasks [Электронный ресурс] / X. Hu, Z. Zhao, S. Wei [и др.] // arXiv preprint 2401.05507. – 2024. – URL: <https://arxiv.org/abs/2401.05507> (дата обращения 17.05.2026).
- [23] Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation / J. Liu, C. S. Xia, Y. Wang, L. Zhang // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 21558–21572.
- [24] LangGraph: A framework for building stateful, multi-actor applications with LLMs [Электронный ресурс] / LangChain AI. – Электрон. дан. – 2024. – URL: <https://langchain-ai.github.io/langgraph/> (дата обращения 17.05.2026).
- [25] LangSmith: Observability and evaluation for LLM applications [Электронный ресурс] / LangChain AI. – Электрон. дан. – 2024. – URL: <https://docs.smith.langchain.com/> (дата обращения 17.05.2026).
- [26] Lin C.-Y. ROUGE: A package for automatic evaluation of summaries / C.-Y. Lin // Text Summarization Branches Out: Proceedings of the ACL-04 Workshop. – Stroudsburg, PA: ACL, 2004. – P. 74–81.
- [27] LiveCodeBench: Holistic and contamination-free evaluation of large language models for code [Электронный ресурс] / N. Jain, K. Han, A. Gu [и др.] // arXiv preprint 2403.07974. – 2024. – URL: <https://arxiv.org/abs/2403.07974> (дата обращения 17.05.2026).
- [28] Liu Z. Dynamic LLM-agent network: an LLM-agent collaboration framework with agent team optimization [Электронный ресурс] / Z. Liu, Y. Zhang, P. Li, Y. Liu, D. Yang // arXiv preprint 2310.02170. – 2023. – URL: <https://arxiv.org/abs/2310.02170> (дата обращения 17.05.2026).
- [29] Magentic-One: A generalist multi-agent system for solving complex tasks [Электронный ресурс] / A. Fourney, G. Bansal, H. Mozannar [и др.] // arXiv preprint 2411.04468. – 2024. – URL: <https://arxiv.org/abs/2411.04468> (дата обращения 17.05.2026).
- [30] Measuring mathematical problem solving with the MATH dataset / D. Hendrycks, C. Burns, S. Kadavath [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2021. – Vol. 34. – P. 2280–2292.

- [31] MetaGPT: Meta programming for a multi-agent collaborative framework / S. Hong, M. Zhuge, J. Chen [и др.] // Proceedings of the International Conference on Learning Representations (ICLR 2024). – 2024.
- [32] Mixture-of-Agents enhances large language model capabilities / J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, J. Zou // Proceedings of the Conference on Language Modeling (COLM 2024). – 2024.
- [33] MMLU-Pro: A more robust and challenging multi-task language understanding benchmark [Электронный ресурс] / Y. Wang, X. Ma, G. Zhang [и др.] // arXiv preprint 2406.01574. – 2024. – URL: <https://arxiv.org/abs/2406.01574> (дата обращения 17.05.2026).
- [34] Mosqueira-Rey E. Human-in-the-loop machine learning: a state of the art / E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, Á. Fernández-Leal // Artificial Intelligence Review. – 2023. – Vol. 56, no. 4. – P. 3005–3054.
- [35] NetSafe: Exploring the topological safety of multi-agent networks against adversarial attacks [Электронный ресурс] / M. Yu, S. Wang, G. Zhang, H. Li // arXiv preprint 2410.15686. – 2024. – URL: <https://arxiv.org/abs/2410.15686> (дата обращения 17.05.2026).
- [36] Ouyang L. Training language models to follow instructions with human feedback / L. Ouyang, J. Wu, X. Jiang [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2022. – Vol. 35. – P. 27730–27744.
- [37] ReAct: Synergizing reasoning and acting in language models / S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, Y. Cao // Proceedings of the International Conference on Learning Representations (ICLR 2023). – 2023.
- [38] Reflexion: Language agents with verbal reinforcement learning / N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 8634–8652.
- [39] RouteLLM: Learning to route LLMs with preference data [Электронный ресурс] / I. Ong, A. Almahairi, V. Wu [и др.] // arXiv preprint 2406.18665. – 2024. – URL: <https://arxiv.org/abs/2406.18665> (дата обращения 17.05.2026).
- [40] Scaling large-language-model-based multi-agent collaboration [Электронный ресурс] / C. Qian, Z. Ye, W. Liu [и др.] // arXiv preprint 2406.07155. – 2024. – URL: <https://arxiv.org/abs/2406.07155> (дата обращения 17.05.2026).
- [41] Self-Refine: Iterative refinement with self-feedback / A. Madaan, N. Tandon, P. Gupta [и др.] // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 46534–46594.
- [42] Tipping the dominos: Topology-aware multi-hop attacks on LLM-based multi-agent systems [Электронный ресурс] / R. Liang, J. Ye, Z. Chen, Q. Zhu // arXiv preprint 2412.04129. – 2024. – URL: <https://arxiv.org/abs/2412.04129> (дата обращения 17.05.2026).
- [43] Training verifiers to solve math word problems [Электронный ресурс] / K. Cobbe, V. Kosaraju, M. Bavarian [и др.] // arXiv preprint 2110.14168. – 2021. – URL: <https://arxiv.org/abs/2110.14168> (дата обращения 17.05.2026).
- [44] Tree of Thoughts: Deliberate problem solving with large language models / S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, K. Narasimhan // Advances in Neural Information Processing Systems. – Red Hook, NY: Curran Associates, 2023. – Vol. 36. – P. 11809–11822.

Приложение А Функциональная схема системы

В настоящем приложении приведена развернутая функциональная схема программной системы, описанной в главе 3. Схема изображает движение управления и данных между основными архитектурными блоками от поступления входных данных эксперимента до получения результирующих метрик и журналов. На рисунке А.1 сплошными стрелками показано движение управления, штриховыми — обмен сообщениями между агентами, точечными — потоки записи в подсистему наблюдаемости и хранения.

Пояснение к схеме

Поток выполнения начинается с поступления конфигурации эксперимента в формате YAML и описания задачи T из выбранного корпуса. Оркестратор эксперимента (`atm.experiment`) инициализирует общее состояние графа и передает управление слою адаптации. Маршрутизатор фаз принимает решение о текущей фазе решения, передает результат маршрутизатору топологий, который выбирает активную коммуникационную топологию. В работе одновременно действует ровно одна из пяти базовых топологий, а адаптивный режим реализуется через последовательное переключение между ними по решению маршрутизатора.

Активная топология определяет порядок активации агентов и протокол обмена сообщениями между ними. Любая из шести ролей агентов (планировщик, исследователь, исполнитель, критик, дебатер, координатор) может быть подключена к любой топологии в соответствии с правилами раздела 2.2. Агенты обращаются за внешними службами — инструментами и Docker-песочницей для исполнения кода, оболочкой над языковыми моделями для генерации текста, шлюзом взаимодействия с человеком.

Все события системы (вызовы языковых моделей, выставление сигналов, переходы между фазами, переключения топологий, обращения к человеку, срабатывания защитных правил маршрутизатора) перехватываются подсистемой наблюдаемости и направляются в два хранилища — реляционную

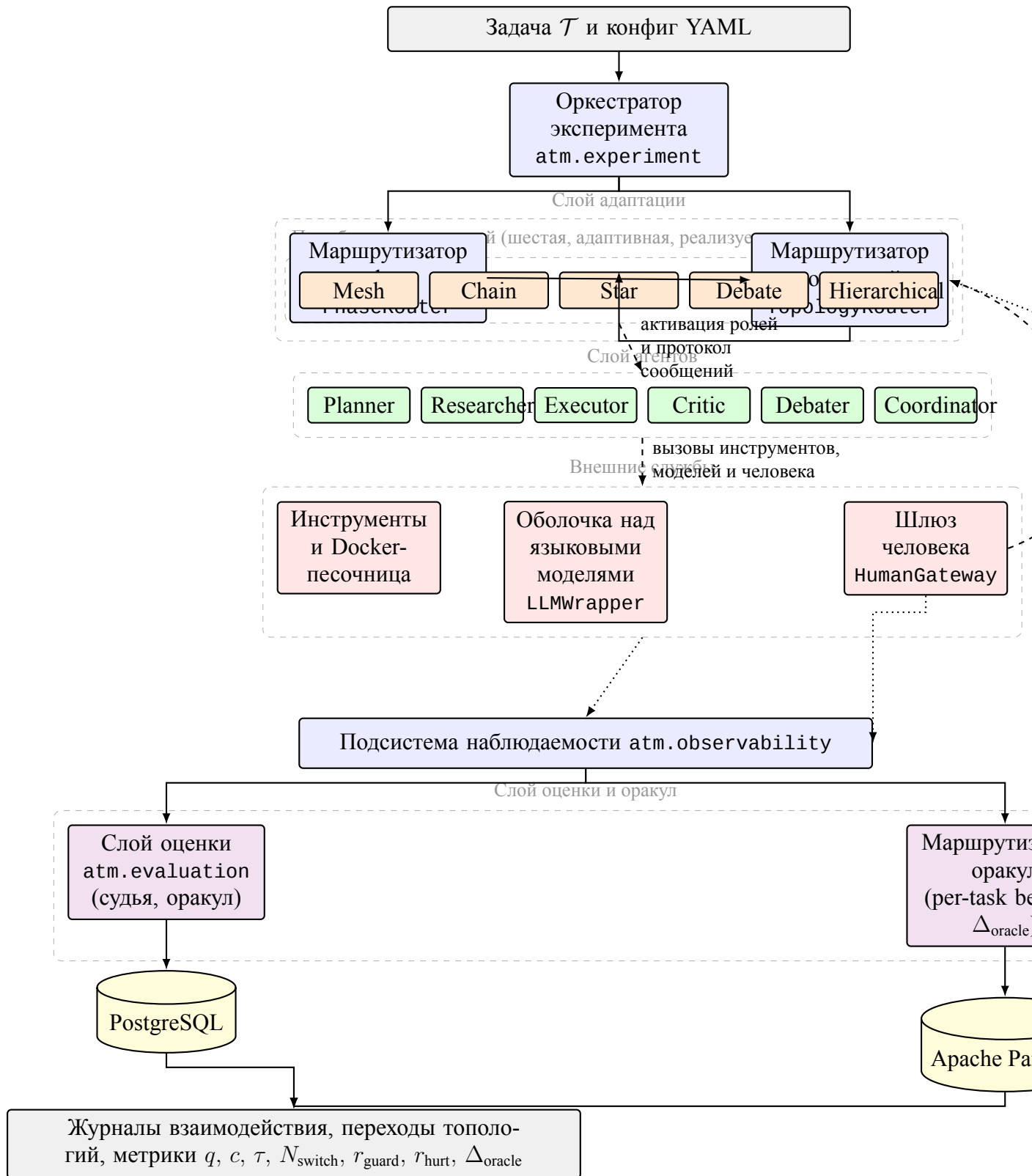


Рисунок А.1. Функциональная схема программной системы. Сплошные стрелки — передача управления и записи событий в хранилище; штриховые — обмен сообщениями между блоками и управляющие сигналы от человека; точечные — поток оракульной таблицы от оракула к маршрутизатору топологий. Цветовое кодирование — серый цвет соответствует точкам входа и выхода, синий — управляющим компонентам, оранжевый — активным коммуникационным топологиям, зеленый — слою агентов, красный — внешним службам исполнения, фиолетовый — слою оценки качества и оракулу, желтый — подсистеме хранения.

базу для метаданных и колоночные файлы Apache Parquet для объемных журналов. На финальной стадии результаты прогона становятся доступными для постобработки слою анализа `atm.analysis`.

Обратный канал от шлюза человека к маршрутизатору топологий обозначает поступление управляющих сигналов от участника-человека к адаптивному слою. Через этот канал человек, действующий в роли координатора или наблюдателя, может форсировать переключение топологии или досрочно завершить фазу проверки. Точечная связь от шлюза к подсистеме наблюдаемости отвечает за регистрацию взаимодействий с человеком, на основе которой вычисляются метрики r_{guard} и r_{hurt} .

Приложение Б Глоссарий

В приложении приведены определения ключевых терминов работы. Термины сгруппированы по тематическим блокам.

Мульти-агентные системы и языковые модели

Большая языковая модель (от англ. Large Language Model, LLM) — глубокая нейронная сеть, обученная на больших текстовых корпусах задаче языкового моделирования (предсказание следующего токена) с возможной дополнительной настройкой под инструкции и обратную связь от человека. **Мульти-агентная система** (от англ. Multi-Agent System, MAS) — программная система из нескольких автономных агентов, взаимодействующих между собой и с внешней средой. **Агент** — программная сущность с фиксированной ролью, системной подсказкой, набором инструментов и собственным состоянием. **Системная подсказка** — постоянная инструкция, задающая агенту роль, ограничения и стиль ответов; не меняется в пределах одного запуска. **Скрэтчпад** (от англ. scratchpad) — локальный приватный контекст агента со скользящим окном истории сообщений и автоматическим сжатием при переполнении. **Рабочий агент** (worker) — агент, непосредственно участвующий в решении задачи; противопоставляется управляющим компонентам. **Судья оценки качества** (LLM-as-judge) — внешняя языковая модель из другого семейства весов, оценивающая качество финального ответа системы с самосогласованностью. **Симулятор человека** — языковая модель в роли участника с системной подсказкой, зависящей от заданной роли. **Состояние графа** (graph state) — общий контейнер контекстов агентов, текущей фазы, активной топологии и журнала переходов. **Бюджет вычислений** — трехуровневый ограничитель стоимости вызовов языковых моделей — на отдельный вызов, на полный прогон и на грид-эксперимент.

Коммуникационные топологии

Коммуникационная топология — направленный граф допустимых маршрутов обмена сообщениями между агентами и порядка их активации. **Топология Star** — центральный координатор последовательно вызывает специализированных работников и агрегирует их ответы. **Топология Chain** — линейный конвейер из трех агентов с обратными связями для доработки. **Топология Mesh** — полносвязная конфигурация с общей шиной сообщений и циклическим обменом. **Топология Debate** — два оппонента параллельно генерируют альтернативные решения, судья выбирает победителя. **Топология Hierarchical** — двухуровневая иерархия из координатора верхнего уровня и двух подкоманд. **Адаптивная метатопология** — конфигурация с выбором активной базовой топологии на каждой итерации в

зависимости от фазы и сигналов агентов; авторская разработка.

Фазы и маршрутизаторы

Фаза решения задачи — состояние конечного автомата из четырех значений (планирование, исполнение, проверка, завершение). **Конечный автомат** (от англ. Finite-State Machine, FSM) — математическая модель с конечным множеством состояний и переходов между ними по входным символам. **Сигнал агента** — булев индикатор готовности к следующей фазе, тупикового состояния, отказа критика. **Маршрутизатор фаз** (PhaseRouter) — компонент принятия решения о переходе процесса в следующую фазу; реализован в трех режимах — правилковый, на основе языковой модели и оракульный. **Маршрутизатор топологий** (TopologyRouter) — компонент смены активной базовой топологии внутри адаптивной мета-топологии. **Защитные правила переключения** (SwitchGuards) — ограничения против частого осцилляционного переключения (минимальное число итераций в топологии, интервал между сменами, лимиты за фазу и за запуск). **Ворота передачи состояния** (TransitionGate) — компонент применения правил передачи состояния при смене активной топологии и регистрации в журнале.

Человек в контуре управления

Человек в контуре управления (от англ. Human-in-the-Loop, HITL) — парадигма, при которой автоматизированная система обращается к человеку за решением, оценкой или подтверждением. **Шлюз взаимодействия с человеком** (HumanGateway) — программный протокол с идемпотентностью повторных вызовов по паре идентификаторов запуска и запроса. **Роль координатора** — человек управляет планом и делегированием задач агентам. **Роль рецензента** — человек проверяет промежуточные результаты и инициирует доработку. **Роль судьи** — человек выносит финальный вердикт при наличии альтернативных решений. **Роль равноправного участника** (peer) — человек вносит собственные сообщения в общую шину наравне с агентами. **Роль наблюдателя** (monitor) — человек пассивно следит за процессом и вмешивается только в критических случаях. **Маршрутизатор ролей** (RoleRouter) — компонент динамического выбора активной роли человека по фазе и наблюдаемому состоянию. **Шкала NASA-TLX** (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA) — стандартный инструмент субъективной оценки когнитивной нагрузки по шести подшкалам. **Оракул маршрутизатора** — идеализированный маршрутизатор, использующий апостериорные данные о качестве каждой топологии на отложенной выборке для оценки верхней границы достижимого качества при заданном пространстве конфигураций.

Метрики

Качество решения (q) — агрегированный показатель в $[0, 1]$; конкретная формула зависит от класса задач. **Полная стоимость прогона** (c) — сумма стоимостей вызовов языковых моделей в долларах США за один прогон. **Время прогона** (τ) — полное время выполнения по настенным часам. **Доля отказов** (r_{fail}) — доля прогонов со статусом, отличным от завершенного. **Число переключений топологии** (N_{switch}) — фактическое число смен активной базовой топологии за прогон адаптивной мета-топологии. **Доля заблокированных решений маршрутизатора** (r_{guard}) — доля решений, заблокированных защитными правилами. **Стоимостный вклад маршрутизатора** (γ) — доля бюджета, потраченная на собственные вызовы маршрутизатора. **Доля регрессий качества** (r_{hurt}) — доля задач с качеством хуже лучшей статической конфигурации; ключевой индикатор безопасности адаптации. **Разрыв до оракула** (Δ_{oracle}) — разность между качеством оракул-выбора лучшей статической топологии для каждого корпуса (per-task best-of) и качеством фактического маршрутизатора; верхняя граница достижимого качества при заданном пространстве статических топологий. **Прокси-метрика когнитивной нагрузки** (L_{cog}) — агрегированный показатель из числа обращений, объема контекста и средней задержки ответа симулятора.

Инфраструктура

LangGraph — библиотека Python для построения и исполнения направленных графов состояния с асинхронными вызовами и чекпойнтером. **Чекпойнтер** — механизм сохранения промежуточного состояния графа для возобновления прерванных запусков. **Apache Parquet** — колоночный формат хранения структурированных данных общего назначения; в работе используется для журналов взаимодействия агентов. **Грид-эксперимент** — запуск серии прогонов по декартову произведению значений нескольких параметров с параллельным исполнением. **Прогон** (run) — один полный цикл выполнения мульти-агентной системы; элементарная единица измерения.

Эксперимент и статистика

Корпус задач — размеченное множество тестовых задач одного класса с автоматической процедурой оценки. **Воронка экспериментов** — последовательность экспериментов с сужением пространства конфигураций по результатам предыдущих. **Подтверждающий прогон** — дополнительный эксперимент на отличной языковой модели для проверки независимости выводов от семейства весов. **Дисперсионный анализ (ANOVA)** — метод проверки гипотезы о равенстве средних в нескольких группах. **Поправка Бенджамини–Хохберга** — процедура контроля доли ложных открытий при множественном сравнении. **Коэффициент эффекта Коэна** (d) — стандартизированная мера величины различия двух средних.

Метод бутстрапа — непараметрический метод оценивания распределения статистики через ресэмплирование выборки; применяется для доверительных интервалов с уровнем 95 % и тысячей повторений. **Доверительный интервал** — интервал, покрывающий истинное значение параметра распределения с заданной вероятностью. **Оракул per-task best-of** — референсная процедура построения верхней границы качества адаптации: для каждого корпуса задач выбирается статическая топология с максимальным средним качеством на этом корпусе по базовому эксперименту. **Конфигурация системы** — набор значений всех переменных эксперимента; различают статические (с фиксированной топологией) и адаптивные (с переключением).

Приложение В Состав программного репозитория

В приложении приведено краткое описание состава программного репозитория проекта. Состав фиксирован по состоянию идентификатора фиксации, указанного в приложении Г.

Корневая структура

src/atm/ — основной пакет фреймворка, тринадцать функциональных слоев. **conf/** — декларативная конфигурация экспериментов и компонентов в формате YAML. **scripts/** — самостоятельные скрипты обслуживания экспериментов и сборки производных артефактов. **notebooks/** — шаблон Jupyter-ноутбука для аналитической постобработки результатов прогонов. **pyproject.toml** — декларация пакета, зависимостей и точки входа atm. **README.md** — точка входа в документацию проекта и инструкция по сборке среды.

Слой пакета **src/atm/**

atm.core — контейнеры состояния и базовые типы (AgentState, SharedState, GraphState, Phase, HumanRole). **atm.llm** — оболочка над провайдерами языковых моделей, ценообразование, бюджетные ограничители, детерминированный FakeLLM, фабрика build_llm. **atm.tools** — реестр инструментов и обертка над Docker-песочницей. **atm.agents** — базовый класс Agent и пять специализированных ролей плюс координатор. **atm.topology** — шесть реализаций топологий (star.py, chain.py, mesh.py, debate.py, hierarchical.py, adaptive.py) поверх общего протокола Topology. **atm.phases** — конечный автомат фаз, маршрутизатор топологий, защитные правила переключения, ворота передачи состояния. **atm.human** — слой человека в контуре управления (от англ. Human-in-the-Loop, HITL) — протокол HumanGateway, реализации шлюзов, маршрутизатор ролей, политика тайм-аутов. **atm.storage** — объектно-реляционное отображение, модели таблиц PostgreSQL, асинхронные сессии, писатель Parquet, обертка над чекпойнтером LangGraph, миграции alembic/. **atm.observability** — перехватчик событий LangGraph, сериализация, обертки над логгером. **atm.tasks** — загрузчики и оценщики четырех корпусов (HumanEval, GSM8K, CommonGen, InfiAgent-DABench). **atm.evaluation** — судья оценки качества, оракулы достоверности, агрегатор шкалы NASA-TLX. **atm.experiment** — схемы конфигов на Pydantic, загрузчик OmegaConf с расширением по сетке, командная строка atm на Typer. **atm.analysis** — загрузчики результатов, агрегаторы метрик, генератор графиков, построитель оракула.

Конфигурационные файлы **conf/**

conf/experiments/ — полные конфигурации экспериментов E1–E4, подтверждающих прогонов и проверок работоспособности. **conf/agents/** — системные подсказки шести ролей агентов. **conf/topology/** — параметры по умолчанию для шести топологий. **conf/human/** — параметры шлюза HITL и системные подсказки симулятора. **conf/model/** — параметры языковых моделей. **conf/tasks/** — загрузчики корпусов и стратифицированные выборки. **conf/oracle/** — конфигурации построения оракула. **conf/evaluation/** — параметры судьи и агрегаторов. **conf/sandbox/** — параметры Docker-песочницы и профиль системных вызовов.

Скрипты и ноутбуки

scripts/gen_analysis_notebook.py — генератор аналитического ноутбука из шаблона. **scripts/derive_seccomp.py** — сборка профиля системных вызовов для песочницы. **notebooks/anal** — шаблонный ноутбук из шести секций для постобработки и визуализации результатов прогона.

Приложение Г Открытый исходный код

В соответствии с пунктом 5.5 Правил подготовки и защиты выпускных квалификационных работ образовательной программы «Прикладной анализ данных и искусственный интеллект» полная программная реализация настоящего исследования размещена в открытом доступе. Среди допустимых методичкой платформ (GitHub, GitLab, Bitbucket, Яндекс.Диск) выбран GitHub — благодаря наличию встроенного механизма версионирования с уникальными хеш-идентификаторами коммитов, что обеспечивает однозначное соответствие текста работы конкретному состоянию исходного кода, поддержке стандартных открытых лицензий и широкой распространенности платформы в академической и инженерной практике.

Адрес репозитория <https://github.com/cactus-tim/adaptive-topologies-m>

Идентификатор фиксации коммит 058271a39a0e (полный хеш — 058271a39a0e4 ветки main по состоянию на дату сдачи работы.

Лицензия исходный код распространяется по лицензии MIT (текст лицензии — в файле LICENSE репозитория).

Сборка среды полная инструкция по установке зависимостей, развертыванию базы данных PostgreSQL и проверке корректности сборки приведена в файле README.md.

Воспроизведение результатов конфигурации экспериментов главы 4 размещены в директории conf/experiments/. Запуск одиночного эксперимента — `atm run -config conf/experiments/e1.yaml`; запуск грида — `atm grid -config conf/experiments/e1.yaml`. Проверки работоспособности на детерминированном FakeLLM — `atm replay -record tests/fixtures/replay/`.

Служебные артефакты кешы, журналы прогонов и временные файлы в репозиторий не включены; они генерируются локально в директориях .cache/ и runs/, исключенных из контроля версий через .gitignore.

Согласие на публикацию работы

В соответствии с пунктом 5.6.3 Правил подготовки и защиты выпускных квалификационных работ согласие автора на публикацию настоящей выпускной квалификационной работы на портале Национального исследовательского университета «Высшая школа экономики» оформлено в виде QR-кода системы «LMS-Антиплагиат», прикрепленного к итоговому отчету о проверке работы. Указанный отчет передан в составе пакета документов в Государственную экзаменационную комиссию.